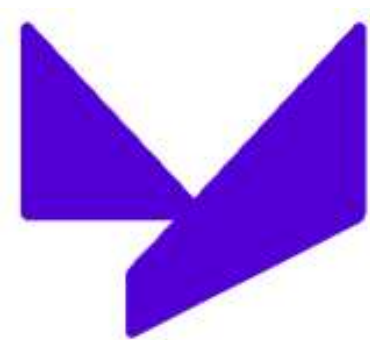




Уфимский университет науки и технологий

Институт информатики, математики и робототехники

Кафедра вычислительной математики и кибернетики

Методы искусственного интеллекта

Д.т.н., профессор кафедры ВМиК Ризванов Дмитрий Анварович

Уфа - 2025

Литература

1. Рассел С., Норвиг П. Искусственный интеллект: современный подход. – М.: Вильямс, 2006. – 1408 с.
2. Люггер Дж. Искусственный интеллект: стратегии и методы решения сложных проблем. – М.: Вильямс, 2005. – 864 с.
3. Лорьер Ж.-Л. Системы искусственного интеллекта. –М.: Мир, 1991. – 568 с.
4. Попов Д. В., Ризванов Д.А. Системы искусственного интеллекта. Эвристический поиск и инженерия знаний: [учебное пособие для студентов высших учебных заведений, обучающихся по специальности 230105 - "Программное обеспечение вычислительной техники и автоматизированных систем"], ФГБОУ ВПО УГАТУ - Уфа: УГАТУ, 2012 - 117 с.

Предыстория ИИ

- Философия основы ИИ, сформулировав идеи, что мозг в определенных отношениях напоминает машину, оперирует знаниями, закодированными на внутреннем языке, мышление может использоваться для выбора наилучших действий.
- Математика инструментальные средства для манипулирования высказываниями, обладающими логической достоверностью, а также недостоверными вероятностными высказываниями. Заложили основу формирования рассуждений об алгоритмах.
- Экономика формализация проблемы принятия решений, максимизирующих ожидаемый выигрыш.
- Психология идея «любая теория познания должна напоминать компьютерную программу», она должна подробно описывать механизм обработки информации, с помощью которого может быть реализована некоторая познавательная функция.
- Вычислительная техника быстрые мощные компьютеры.
- Теория управления проектирование устройств, которые действуют оптимально на основе обратной связи.
- Лингвистика представление знаний, обработка естественного языка

Краткая история ИИ

- 1943 McCulloch & Pitts: Boolean circuit model of brain
- 1950 Turing's "Computing Machinery and Intelligence"
- 1956 Dartmouth meeting: "Artificial Intelligence" adopted
- 1952—69 Look, Ma, no hands!
- 1950s Early AI programs, including Samuel's checkers program, Newell & Simon's Logic Theorist, Gelernter's Geometry Engine
- 1965 Robinson's complete algorithm for logical reasoning
- 1966—73 AI discovers computational complexity
Neural network research almost disappears
- 1969—79 Early development of knowledge-based systems
- 1980-- AI becomes an industry
- 1986-- Neural networks return to popularity
- 1987-- AI becomes a science
- 1995-- The emergence of intelligent agents

Что такое ИИ?

Системы, которые думают подобно людям

“Новое захватывающее направление работ по созданию компьютеров, способных думать, . . . машин, обладающих разумом, в полном и буквальном смысле этого слова.” (Haugeland, 1985)

“[Автоматизация] действий, которые мы ассоциируем с человеческим мышлением, т.е. таких действий, как принятие решений, решение задач, обучение . . .” (Bellman, 1978)

Системы, которые действуют подобно людям

“Искусство создания машин, которые выполняют функции, требующие интеллектуальности при их выполнении людьми.” (Kurzweil, 1990)

“Наука о том, как научить компьютеры делать то, в чем люди в настоящее время их превосходят.” (Rich and Knight, 1991)

Системы, которые думают рационально

“Изучение умственных способностей с помощью вычислительных моделей.” (Charniak and McDermott, 1985)

“Изучение таких вычислений, которые позволяют чувствовать, рассуждать и действовать.” (Winston, 1992)

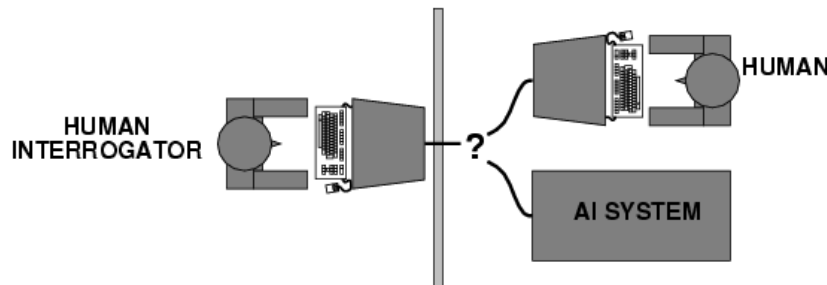
Системы, которые действуют рационально

“Вычислительный интеллект – это наука о проектировании интеллектуальных агентов.” (Poole *et al.*, 1998)

“Искусственный интеллект . . . – это наука, посвященная изучению интеллектуального поведения артефактов.” (Nilsson, 1998)

Системы, которые действуют подобно людям

- Turing (1950) "Computing machinery and intelligence":



Возможности, которыми должен обладать компьютер:

- **средства обработки текстов на естественном языке**, позволяющие успешно общаться с компьютером (например, на англ. языке);
- **средства представления знаний**, с помощью которых компьютер может записать в память то, что он узнает или прочитает;
- **средства автоматического формирования логических выводов**, обеспечивающие возможность использовать хранимую информацию для поиска ответов на вопросы и вывода новых заключений;
- **средства машинного обучения**, которые позволяют приспосабливаться к новым обстоятельствам, а также обнаруживать и экстраполировать признаки стандартных ситуаций;
- **машинное зрение** для восприятия объектов;
- **средства робототехники** для манипулирования объектами и перемещения в пространстве.

Системы, которые думают подобно людям

Существует 3 способа узнать, как думает человек:

- интроспекция — попытка проследить за ходом собственных мыслей;
- психологические эксперименты — наблюдение человека в действии;
- через сканирование мозга – наблюдение за мозгом в действии.

Аллен Ньюэлл и Герберт Саймон “General Problem Solver” (GPS, 1961), не стремились к тому, чтобы эта программа правильно решала поставленные задачи. В большей степени их заботило, чтобы запись этапов проводимых ею рассуждений совпадала с регистрацией рассуждений людей, решающих такие же задачи.

Когнитология – междисциплинарная область, в которой совместно используются компьютерные модели, взятые из искусственного интеллекта, и экспериментальные методы, взятые из психологии, для разработки точных и обоснованных теорий работы человеческого мозга.

Системы, которые думают рационально

Аристотель – законы «правильного мышления».

Его **силлогизмы** стали образцом для создания процедур доказательства, которые всегда позволяют прийти к правильным заключениям, если даны правильные предпосылки.

Основа этих исследований – предположение, что такие законы мышления управляют работой ума. На их основе развивается **логика**.

В 19 веке была разработана точная система логических обозначений для утверждений о предметах любого рода, которые встречаются в мире, и об отношениях между ними (сравните с обычной системой арифметических обозначений, которая предназначена в основном для формирования утверждений о равенстве и неравенстве чисел!!!).

В 1965 г. были разработаны программы, которые могли в принципе решить любую разрешимую проблему, описанную в системе логических обозначений. Если же решение не существует, программа может так и не остановиться в процессе его поиска.

Проблемы:

- довольно сложно любые неформальные знания выразить в формальных терминах, требуемых для системы логических обозначений, особенно если эти знания не являются полностью достоверными.
- есть большая разница между решением проблемы «в принципе» и решением проблемы на практике.

Системы, которые действуют рационально

Агентом считается все, что действует («агент» произошло от латинского «*agere*», действовать). **Рациональный агент** – это агент, который действует таким образом, чтобы можно было достичь наилучшего результата или, в условиях неопределенности, наилучшего ожидаемого результата.

Все навыки, необходимые для прохождения теста Тьюринга, позволяют также осуществлять рациональные действия.

Представление знаний и логический вывод позволяет агентам получать хорошие решения.

Необходимо уметь формировать понятные предложения на **естественном языке**, поскольку в сложный социум принимают только тех, кто способен правильно высказывать свои мысли.

Необходимо **учиться** не только ради приобретения эрудиции, но и в связи с тем, что лучшее представление о том, как устроен мир, позволяет вырабатывать более эффективные стратегии действий в этом мире.

Нужно обладать способностью к **зрительному восприятию** не только потому, что процесс визуального наблюдения позволяет получить удовольствие, но и потому, что зрение подсказывает, чего можно достичь с помощью определенного действия (например, быстрее всех получить лакомый кусочек).

Системы, которые действуют рационально

Преимущества подхода:

- более общий по сравнению с подходом на основе «законов мышления», поскольку правильный логический вывод – это просто один из нескольких возможных механизмов достижения рациональности.
- более перспективный для научной разработки по сравнению с подходами, основанными на изучении человеческого поведения или человеческого мышления, поскольку стандарт рациональности четко определен и полностью обобщен.

Различные подходы к построению систем ИИ

- Логический
- Структурный
- Эволюционный
- Имитационный
- Агентно-ориентированный подход

Логический подход

Основа - Булева алгебра и исчисление предикатов.

Практически каждая система ИИ, построенная на логическом принципе, представляет собой машину доказательства теорем. При этом исходные данные хранятся в базе данных в виде аксиом, правила логического вывода как отношения между ними. Каждая такая машина имеет блок генерации цели, и система вывода пытается доказать данную цель как теорему. Если цель доказана, то трассировка примененных правил позволяет получить цепочку действий, необходимых для реализации поставленной цели.

- **Отсутствие выразительности алгебры высказываний.**

Добиться большей выразительности позволяет **нечеткая логика**. Основным ее отличием является то, что правдивость высказывания может принимать в ней кроме да/нет (1/0) еще и промежуточные значения — не знаю (0.5), пациент скорее жив, чем мертв (0.75), пациент скорее мертв, чем жив (0.25). Данный подход больше похож на мышление человека, поскольку он на вопросы редко отвечает только да или нет.

- **Большая трудоемкость**, поскольку во время поиска доказательства возможен полный перебор вариантов.

Поэтому данный подход требует эффективной реализации вычислительного процесса, и хорошая работа обычно гарантируется при сравнительно небольшом размере базы данных.

Структурный подход

Попытки построения ИИ путем моделирования структуры человеческого мозга.

Одной из первых таких попыток был перцептрон Френка Розенблатта.

Основной моделируемой структурной единицей в перцептронах (как и в большинстве других вариантов моделирования мозга) является нейрон.

Позднее возникли и другие модели, которые известны под термином "нейронные сети" (НС). Эти модели различаются по строению отдельных нейронов, по топологии связей между ними и по алгоритмам обучения.

Для моделей, построенных по мотивам человеческого мозга характерны:

- **не слишком большая выразительность;**
- **легкое распараллеливание алгоритмов;**
- **высокая производительность параллельно реализованных НС.**

Нейронные сети работают даже при условии неполной информации об окружающей среде, то есть как и человек, они на вопросы могут отвечать не только "да" и "нет" но и "не знаю точно, но скорее да" (схожесть с человеческим мозгом).

Эволюционный подход

Основное внимание уделяется построению начальной модели и правилам, по которым она может изменяться (эволюционировать). Причем модель может быть составлена по самым различным методам, это может быть и НС и набор логических правил и любая другая модель. После этого мы включаем компьютер и он, на основании проверки моделей отбирает самые лучшие из них, на основании которых по самым различным правилам генерируются новые модели, из которых опять выбираются самые лучшие и т. д.

В принципе можно сказать, что эволюционных моделей как таковых не существует, существует только эволюционные алгоритмы обучения, но модели, полученные при эволюционном подходе имеют некоторые характерные особенности, что позволяет выделить их в отдельный класс.

Таковыми особенностями являются:

- **перенесение основной работы разработчика с построения модели на алгоритм ее модификации**
- **полученные модели практически не сопутствуют извлечению новых знаний о среде, окружающей систему ИИ, то есть она становится как бы вещью в себе.**

Имитационный подход

Это классический подход для кибернетики с одним из ее базовых понятий — "черным ящиком".

Объект, поведение которого имитируется, как раз и представляет собой такой "черный ящик". Нам не важно, что у него и у модели внутри и как он функционирует, главное, чтобы наша модель в аналогичных ситуациях вела себя точно так же.

Моделируется свойство человека — **способность копировать то, что делают другие, не вдаваясь в подробности, зачем это нужно**. Зачастую эта способность экономит ему массу времени, особенно в начале его жизни.

Основной недостаток - **низкая информационная способность большинства моделей**.

Агентно-ориентированный подход

Основан на использовании **интеллектуальных (рациональных) агентов**.

Интеллект — это вычислительная часть (грубо говоря, планирование) способности достигать поставленных перед интеллектуальной машиной целей. Сама такая машина будет интеллектуальным агентом, воспринимающим окружающий его мир с помощью датчиков и способной воздействовать на объекты в окружающей среде с помощью исполнительных механизмов.

Основной акцент – **на методах и алгоритмах, которые помогут интеллектуальному агенту выживать в окружающей среде при выполнении его задачи**. Здесь значительно сильнее изучаются алгоритмы поиска и принятия решений.

Способ решения задачи основан на **коммуникациях** между агентами.

Современное состояние разработок

- **Robotic vehicles:** A driverless robotic car named STANLEY sped through the rough terrain of the Mojave desert at 22 mph, finishing the 132-mile course first to win the 2005 DARPA Grand Challenge. No hands across America (driving autonomously 98% of the time from Pittsburgh to San Diego)
- **Autonomous planning and scheduling:** NASA's on-board autonomous planning program controlled the scheduling of operations for a spacecraft.
- **Game playing:** Deep Blue defeated the world chess champion Garry Kasparov in 1997
- **Spam fighting:** Each day, learning algorithms classify over a billion messages as spam, saving the recipient from having to waste time deleting what, for many users, could comprise 80% or 90% of all messages.
- **Logistics planning:** During the 1991 Gulf War, US forces deployed an AI logistics planning and scheduling program that involved up to 50,000 vehicles, cargo, and people at a time, and had to account for starting points, destinations, routes, and conflict resolution among all parameters.
- **Machine Translation:** A computer program automatically translates from Arabic to English. None of the computer scientists of the team speak Arabic, but they do understand statistics and machine learning algorithms.
- Proverb solves crossword puzzles better than most humans

Умные машины: до "судного дня" осталось недолго

(<http://top.rbc.ru/economics/31/01/2013/842964.shtml>)

Живое железо

Сегодня появление таких технологических новинок, как распознающий движения контроллер Kinect от Microsoft, на некоторое время "взрывает" среду избалованных продвинутыми девайсами потребителей, но с каждым разом публика все быстрее привыкает к "умным" новациям и ждет, когда контакт с искусственным разумом станет еще теснее. Не сидится спокойно и генераторам IT-идей. "Несмотря на то, что Kinect перенес взаимодействие на физический уровень, многое до сих пор происходит на плоском экране, иногда в 3D. Ввод информации улучшился, а вывод пока не очень. Мы работаем над трехмерными системами отображения на базе различных технологий, в том числе проективных. Нужно впустить компьютерный мир в наш, физический, сделать его более осязаемым. Для этого, однако, нужно распознать не только пользователя, но и пространство вокруг него – тогда мы сможем дополнять реальный мир виртуальными объектами в гораздо более удобной форме", - говорит исследователь Microsoft Research Шахрам Изади.

Его коллега из конкурирующей IBM, вице-президент по инновациям Берни Мейерсон, соглашается, добавляя, что компьютеры в ближайшие годы должны стать еще "умнее", помогать человеку получать информацию, эффективно ее использовать. "Когнитивные вычислительные системы помогут выделять в данных ключевое, успевать за скоростью их распространения, принимать более взвешенные решения, укреплять здоровье и повышать уровень жизни, разнообразить жизнь и устранять любые препятствия и барьеры: географические, языковые, связанные со стоимостью и доступностью услуг", - уверен Б.Мейерсон.

Картинка на ощупь

Уже в ближайшие пять лет, уверены разработчики из IBM, компьютеры совершат мощный прорыв в области имитации органов чувств человека. Известно, что тач-интерфейс является одним из главных трендов в области потребительской электроники. И сегодня ведутся разработки по его совершенствованию. Так, ученые IBM хотят "научить" сенсорный экран передавать тактильные ощущения. Такая фишка может прийти по вкусу любительницам онлайн-шопинга: к привычному рассматриванию фотографии приглянувшегося платья добавиться возможность и "потрогать" его. Проводя пальцами по экрану с фотографией наряда, можно будет почувствовать текстуру материала, из которого он сделан.

Для реализации этой идеи используются технологии чувствительности к тактильным воздействиям, инфракрасному свету и давлению для имитации осязания, а также... стандартная функция вибрации сотового телефона. Она позволяет назначить каждому объекту уникальный набор вибрационных характеристик, который и будет передавать тактильные ощущения.

Пиксель знает

Разработчики IBM сетуют, что компьютеры сегодня понимают смысл изображения только по текстовым тегам и названию (само содержание изображения пока остается для компьютеров за гранью понимания). Но ученые не унывают и работают над тем, чтобы компьютерные системы научись, анализируя пиксели, понимать смысл изображения, как это делает человек.

Возможность компьютеров получать знания из визуальных данных окажет серьезное влияние на здравоохранение, розничную торговлю и сельское хозяйство, уверены в IBM. Так, техника сможет анализировать изображения компьютерной томографии, рентгеновских снимков и результатов УЗИ, при этом распознавая едва различимые или невидимые для человеческого глаза детали.

Ухо искусственного разума

У человеческого слухового аппарата тоже в скором времени появится компьютерный конкурент. В течение пяти лет IT-технологии IBM намерены представить распределенную сеть "умных" датчиков для определения звукового давления и колебаний, а также различения звуковых волн на разных частотах. Такая система сможет интерпретировать звуки, она будет "прислушиваться" к окружающему миру, определяя перемещения или отмечая степень напряжения материалов, что может пригодиться для обеспечения технической безопасности различных сооружений.

При этом, вероятно, все названные системы будут работать во взаимодействии друг с другом, они будут классифицировать и интерпретировать информацию в соответствии с приобретенными знаниями, а при обнаружении новых звуков система будет формулировать выводы, руководствуясь предыдущими знаниями и используя способность распознавать закономерности.

Кроме того, отмечают в IBM, в ближайшие пять лет компьютерные системы смогут фиксировать все характеристики беседы людей и анализировать тон, тембр, неуверенность в голосе, ориентируясь на изменения в эмоциональном настрое.

Компьютер-поварешка

Исследователи IBM уверены: здоровая пища должна быть вкусной! Они разрабатывают компьютерную систему - по сути, цифровые вкусовые рецепторы - способную создавать новые рецепты, действуя по непростому алгоритму. Она будет расщеплять ингредиенты до их молекулярного уровня и составлять химические композиции пищевых соединений с учетом вкусовых и ароматических предпочтений людей.

Система позволит изучать то, как химические вещества взаимодействуют друг с другом, выяснять их вкусовые составляющие на молекулярном уровне и использовать эту информацию в сочетании с моделями восприятия для прогнозирования вкусовой привлекательности продукта, рассказывают в IBM. Наверняка, такая система не только поможет сделать здоровую пищу более приятной, она также удивит необычными комбинациями продуктов.

Лови мое дыхание

...И вот опять простуда. И вроде бы Минздрав предупреждал, и старались обходить стороной заболевшего, но все равно, когда поняли, что простуда накрыла, было уже поздно. Подобная история может окончательно уйти в прошлое уже в ближайшие годы. Специалисты IBM обещают, что в следующие пять лет встроенные в компьютер или мобильный телефон крошечные датчики смогут на ранней стадии определить симптомы простуды или других заболеваний. "Анализируя запахи, биомаркеры и тысячи молекул в дыхании человека и соотнося их с нормой, компьютерные системы будут помогать врачам в диагностике и мониторинге различных заболеваний", - говорят в компании.

Работающие по аналогичному принципу датчики смогут собирать и анализировать данные там, где это считалось недоступным. В частности, компьютерные системы можно будет использовать в сельском хозяйстве для определения запаха и анализа состояния почвы. В городских условиях эта технология может применяться для мониторинга санитарного состояния и уровня загрязнения территорий и помещений, помогая службам города выявлять потенциальные проблемы прежде, чем они нанесут реальный ущерб.

Человек-проводник

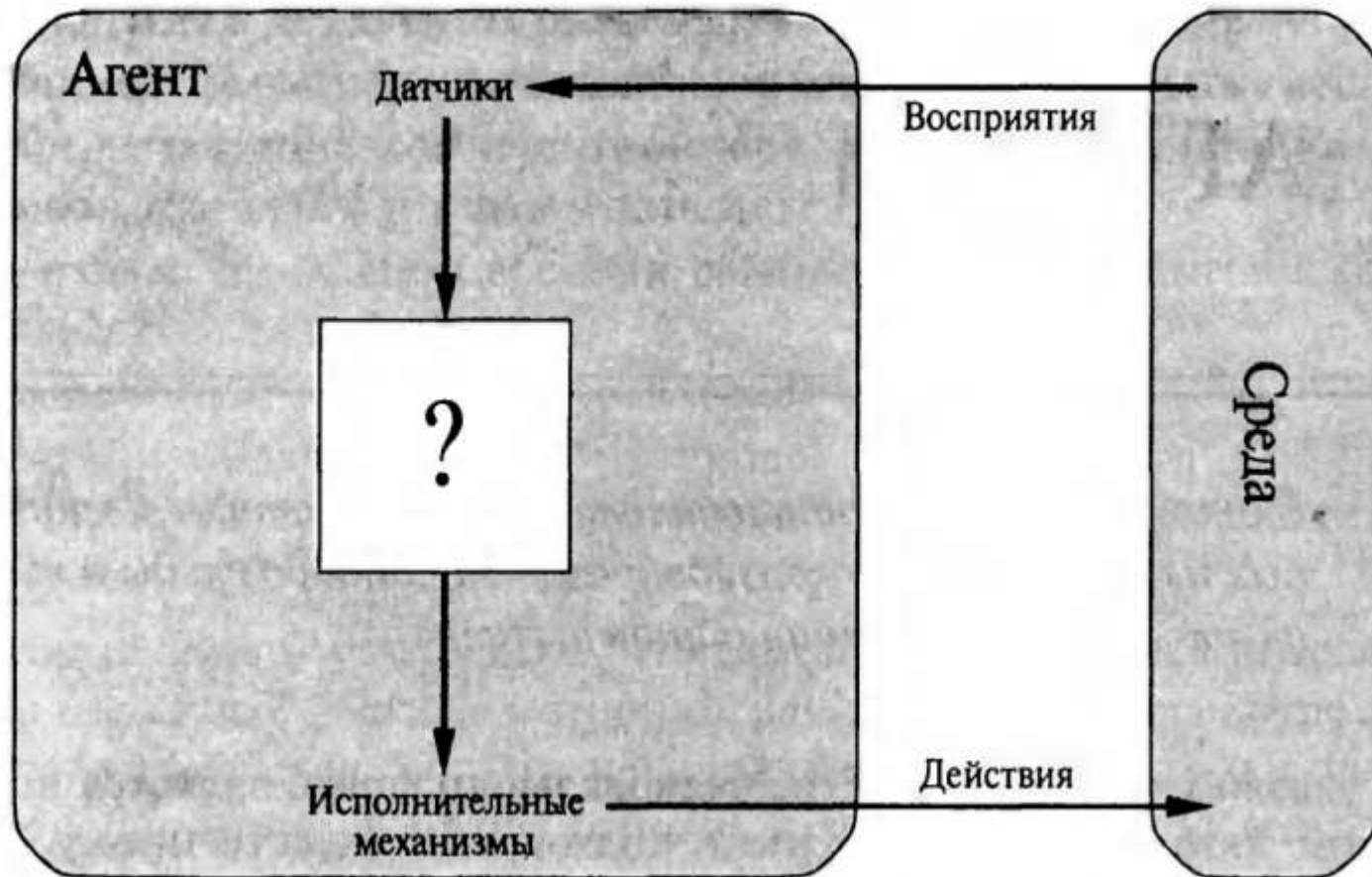
Разработчики IT-инноваций постоянно твердят о сближении виртуального мира и реального, об очеловечивании машины. Но такая конвергенция наводит на мысль и о возможности обратного процесса - компьютеризации самого человека. И эта мысль давно вышла за границы сценариев фантастических фильмов. Компания Ericsson, в свое время создавшая технологию Bluetooth, как раз работает в этом направлении.

"Сегодня мы активно работаем над развитием технологии, которая позволяет обмениваться информацией и связывать устройства через тело человека", - рассказала директор по коммуникациям Ericsson в России, Украине, СНГ и Центральной Азии Анастасия Тимошина. Она пояснила, что речь идет о технологии Connected Me, впервые представленной Ericsson в прошлом году. Connected Me позволяет передавать данные через тело человека на скорости до 10 Мбит/с, но в перспективе можно будет говорить о "разгоне" до 20-40 Мбит/с. А это значит, что уже в недалеком будущем станет возможна передача музыки и фотографий, обмен визитками, электронная оплата покупок в одно касание, пусть даже сегодня это кажется фантастикой.

Интеллектуальные агенты

Агенты и варианты среды

Агентом является все, что может рассматриваться как воспринимающее свою *среду* с помощью *датчиков* и воздействующее на эту среду с помощью *исполнительных механизмов* (например, человек, робот, ПО и т.д.).



Агенты и варианты среды

Термин «*восприятие*» используется для обозначения полученных агентом сенсорных данных в любой конкретный момент времени.

Последовательностью актов восприятия агента называется полная история всего, что было когда-либо воспринято агентом.

Выбор агентом действия в любой конкретный момент времени может зависеть от всей последовательности актов восприятия, наблюдавшихся до этого момента времени. Если существует возможность определить, какое действие будет выбрано агентом в ответ на любую возможную последовательность актов восприятия, то может быть дано более или менее точное определение агента.

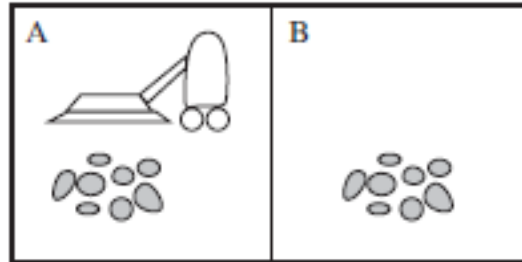
Поведение некоторого агента может быть описано с помощью *функции агента*, которая отображает любую конкретную последовательность актов восприятия на некоторое действие.

Может рассматриваться задача табуляции функции агента, которая описывает любого конкретного агента. Такая таблица – *внешнее описание агента*.

Внутреннее описание состоит в определении того, какая функция агента для данного искусственного агента реализуется с помощью *программы агента*.

Функция агента представляет собой абстрактное математическое описание, а *программа агента* – это конкретная реализация, действующая в рамках архитектуры агента.

Пример. Мир пылесоса



Восприятие:

- местоположение (в каком квадрате он находится);
- состояние квадрата (есть ли мусор в этом квадрате).

Действия: переход влево, вправо, всасывание мусора или бездействие.

Последовательность актов восприятия	Действие
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean] , [A, Clean]	Right
[A, Clean] , [A, Dirty]	Suck
...	...
[A, Clean] , [A, Clean] , [A, Clean]	Right
[A, Clean] , [A, Clean] , [A, Dirty]	Suck
...	...

... ?

Качественное поведение: концепция рациональности

Рациональным агентом является такой агент, который выполняет правильные действия; или более формально, агент, в котором каждая запись в таблице для функции агента заполнена правильно.

выполнение правильных действий?

Показатели производительности

Показатели производительности воплощают в себе критерии оценки успешного поведения агента.

Последовательность действий агента вынуждает среду пройти через последовательность состояний. Если такая последовательность соответствует желаемому, то агент функционирует хорошо.

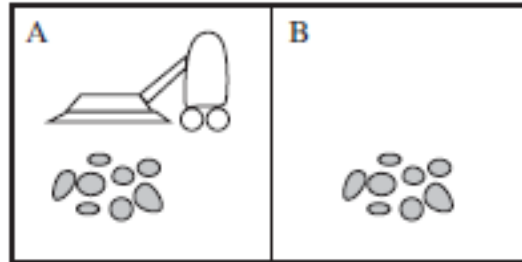
Не может быть одного постоянного показателя, подходящего для всех агентов.



Можно узнать у агента его субъективное мнение о том, насколько он удовлетворен своей собственной производительностью, но некоторые агенты не будут способны ответить, а другие склонны заниматься самообманом («зеленый виноград»).

Поэтому необходимо добиваться применения объективных показателей производительности, и, как правило, проектировщик, конструирующий агента, предусматривает такие показатели.

Агент-пылесос



Показатель производительности???

Рекомендация: лучше всего разрабатывать показатели производительности в соответствии с тем, чего действительно необходимо добиться в данной среде, а не в соответствии с тем, как, по мнению проектировщика, должен вести себя агент.

Философская проблема предпочтительности: что лучше — бесшабашная жизнь со взлетами и падениями или безопасное, но однообразное существование? Что лучше — экономика, в которой каждый живет в умеренной бедности, или такая экономика, в которой одни ни в чем не нуждаются, а другие еле сводят концы с концами?

Рациональность

Зависит от следующих факторов:

- Показатели производительности, которые определяют критерии успеха.
- Знания агента о среде, приобретенные ранее.
- Действия, которые могут быть выполнены агентом.
- Последовательность актов восприятия агента, которые произошли до настоящего времени.

Рациональный агент — это агент, который для каждой возможной последовательности актов восприятия должен выбрать действие, которое, как ожидается, максимизирует его показатели производительности, с учетом фактов, предоставленных данной последовательностью актов восприятия и всех встроенных знаний, которыми обладает агент.

Является ли рациональным агент-пылесос?

Всезнание, обучение и автономность

1. Необходимо различать *рациональность* и *всезнание*.

Рациональность — это максимизация ожидаемой производительности, а **совершенство** — максимизация фактической производительности.

Отказываясь от стремления к совершенству, мы не только применяем к агентам справедливые критерии, но и учитываем реальность.

Если от агента требуют, чтобы он выполнял действия, которые оказываются наилучшими после их совершения, то задача проектирования агента, отвечающего этой спецификации, становится невыполнимой.

2. Определение рационального агента требует, чтобы он не только собирал информацию, но и обучался в максимально возможной степени на тех данных, которые он воспринимает.

3. Рациональный агент должен быть автономным — он должен обучаться всему, что может освоить, для компенсации неполных или неправильных априорных знаний (например, агент-пылесос, который обучается прогнозированию того, где и когда появится дополнительный мусор, безусловно, будет работать лучше, чем тот агент, который на это не способен).

Определение проблемной среды

Факторы, определяющие проблемную среду:

- производительность;
- среда;
- исполнительные механизмы;
- датчики.

PEAS (Performance measure, Environment, Actuators, Sensors)

Пример. Описание PEAS проблемной среды для автоматизированного такси

Тип агента	Показатели производительности	Среда	Исполнительные механизмы	Датчики
Водитель такси	Безопасная, быстрая, комфортная езда в рамках правил дорожного движения, максимизация прибыли	Дороги, другие транспортные средства, пешеходы, климатические факторы	Рулевое управление, акселератор, тормоз, световые сигналы, клаксон, дисплей	Видеокамеры, ультразвуковой дальномер, спидометр, глобальная система навигации и определения положения, одометр, акселерометр, датчики двигателя, клавиатура

Примеры типов агентов и их описаний

Тип агента	Показатели производительности	Среда	Исполнительные механизмы	Датчики
Медицинская диагностическая система	Успешное лечение пациента, минимизация затрат, отсутствие поводов для судебных тяжб	Пациент, больница, персонал	Вывод вопросов, тестов, диагнозов, рекомендаций, направлений	Ввод с клавиатуры симптомов, результатов лабораторных исследований, ответов пациента
Система анализа изображений, полученных со спутника	Правильная классификация изображения	Канал передачи данных от орбитального спутника	Вывод на дисплей результатов классификации определенного фрагмента изображения	Массивы пикселей с данными о цвете
Робот-сортировщик деталей	Процентные показатели безошибочной сортировки по лоткам	Ленточный конвейер с движущимися на нем деталями; лотки	Шарнирный манипулятор и захват	Видеокамера, датчики углов поворота шарниров
Контроллер очистительной установки	Максимизация степени очистки, продуктивности, безопасности	Очистительная установка, операторы	Клапаны, насосы, нагреватели, дисплеи	Температура, давление, датчики химического состава
Интерактивная программа обучения английскому языку	Максимизация оценок студентов на экзаменах	Множество студентов, экзаменационное агентство	Вывод на дисплей упражнений, рекомендаций, исправлений	Ввод с клавиатуры

Свойства проблемной среды

- **Полностью наблюдаемая** или **частично наблюдаемая** (если датчики агента предоставляют ему доступ к полной информации о состоянии среды в каждый момент времени, то такая проблемная среда называется полностью наблюдаемой).
- **Детерминированная** или **недетерминированная** (если следующее состояние среды полностью определяется текущим состоянием и действием, выполненным агентом, то такая среда называется детерминированной; в противном случае она является недетерминированной).
- **Эпизодическая** или **последовательная** (в эпизодической проблемной среде опыт агента состоит из неразрывных эпизодов, каждый эпизод включает в себя восприятие среды агентом, а затем выполнение одного действия. При этом крайне важно то, что следующий эпизод не зависит от действий, предпринятых в предыдущих эпизодах. В эпизодических вариантах среды выбор действия в каждом эпизоде зависит только от самого эпизода. В последовательных вариантах среды текущее решение может повлиять на все будущие решения).
- **Дискретная** или **непрерывная** (различие между дискретными и непрерывными вариантами среды может относиться к состоянию среды, способу учета времени, а также восприятиям и действиям агента).

Свойства проблемной среды

- **Статическая** или **динамическая** (если среда может измениться в ходе того, как агент выбирает очередное действие, то такая среда называется динамической для данного агента; в противном случае она является статической. Если с течением времени сама среда не изменяется, а изменяются показатели производительности агента, то такая среда называется **полудинамической**).
- **Одноагентная** или **мультиагентная** (ключевое различие состоит в том, следует ли или не следует описывать поведение объекта В как максимизирующее личные показатели производительности, значения которых зависят от поведения агента А. Например, в шахматах соперничающая сущность В пытается максимизировать свои показатели производительности, а это по правилам шахмат приводит к минимизации показателей производительности агента А. Таким образом, шахматы — это **конкурентная** мультиагентная среда. А в среде вождения такси, с другой стороны, предотвращение столкновений максимизирует показатели производительности всех агентов, поэтому она может служить примером частично **кооперативной** мультиагентной среды. Она является также частично **конкурентной**, поскольку, например, парковочную площадку может занять только один автомобиль).

Известная или неизвестная. Строго говоря, это различие относится не к окружающей среде самой по себе, а к состоянию знаний агента (или его работчика) о действующих в ней "законах. физики". В известной среде результаты

Свойства проблемной среды

- **Статическая** или **динамическая** (если среда может измениться в ходе того, как агент выбирает очередное действие, то такая среда называется динамической для данного агента; в противном случае она является статической. Если с течением времени сама среда не изменяется, а изменяются показатели производительности агента, то такая среда называется *полудинамической*).
- **Известная или неизвестная.** Строго говоря, это различие относится не к окружающей среде самой по себе, а к состоянию знаний агента (или его разработчика) о действующих в ней "законах. физики". В известной среде результаты (или вероятности исхода, если среда является недетерминированной) определены для всех действий. Очевидно, что, если среда неизвестна, агент должен будет изучить, как она функционирует, чтобы принимать правильные решения. Различие между известными и неизвестными средами вовсе не такое, как между полностью наблюдаемой и частично наблюдаемой средами. Вполне возможно, что известная среда может быть частично наблюдаемой, например в карточном пасьянсе игроку известны все правила, но он еще не видит карт, которые пока не были перевернуты. И наоборот, неизвестная среда может быть полностью наблюдаемой, - в новой видеоигре ее интерфейс может быть представлен на экране полностью, но назначение отдельных его элементов будет неизвестно, пока они не будут опробованы.

Какова среда реального мира?

Полностью наблюдаемая или частично наблюдаемая?

Детерминированная или стохастическая?

Эпизодическая или последовательная?

Статическая или динамическая?

Дискретная или непрерывная?

Одноагентная или мультиагентная?

Частично наблюдаемая

Стохастическая

Последовательная

Динамическая

Непрерывная

Мультиагентная

Примеры проблемных сред и их характеристики

Среда проблемы	Наблюдаемость	Агенты	Тип	Повторяемость	Статичность	Дискретность
Решение кроссворда	Полная	Один	Детерминированная	Последовательная	Статическая	Дискретная
Игра в шахматы с контролем времени	Полная	Много	Детерминированная	Эпизодическая	Полудинамическая	Дискретная
Игра в покер	Частичная	Много	Недетерминированная	Последовательная	Статическая	Дискретная
Игра в нарды	Полная	Много	Недетерминированная	Последовательная	Статическая	Дискретная
Вожделение такси	Частичная	Много	Недетерминированная	Последовательная	Динамическая	Непрерывная
Медицинская диагностика	Частичная	Один	Недетерминированная	Последовательная	Динамическая	Непрерывная
Анализ изображений	Полная	Один	Детерминированная	Эпизодическая	Полудинамическая	Непрерывная
Робот — сборщик деталей	Частичная	Один	Недетерминированная	Эпизодическая	Динамическая	Непрерывная
Система контроля очистки	Частичная	Один	Недетерминированная	Последовательная	Динамическая	Непрерывная
Система обучения английскому языку	Частичная	Много	Недетерминированная	Последовательная	Динамическая	Дискретная

Структура агентов

Агент = Архитектура + Программа

```
function Table-Driven-Agent(percept) returns действие action  
  static: percepts, последовательность актов восприятия,  
           первоначально пустая  
           table, таблица действий, индексированная по  
           последовательностям актов восприятия и  
           полностью заданная с самого начала  
  
  добавить результаты восприятия percept к концу  
    последовательности percepts  
  action ← Lookup(percepts, table)  
  return action
```

Проблема с табличным описанием поведения агента

Если P – множество актов восприятия, T – срок существования агента (общее количество актов восприятия, которое может быть им получено), то поисковая таблица будет содержать $\sum_{t=1}^T |P|^t$ записей.

Рассмотрим случай автоматизированного такси: визуальные входные данные с одной видеокамеры (обычно их восемь) поступают со скоростью примерно 70 Мбайт/с (30 кадров в секунду, в кадре 1080 x 720 пикселей с 24 битами цветовой информации). На один час езды потребуется справочная таблица более чем с $10^{600\,000\,000}$ записей! Даже для игры в шахматы (крошечного, хорошо изученного фрагмента реального мира) поисковая таблица должна содержать не менее 10^{150} записей. Для сравнения, количество атомов в наблюдаемой части Вселенной оценивается менее чем в 10^{80} .

Ошеломляющий размер этих таблиц означает, что:

- а) ни один физический агент в этой Вселенной не будет иметь достаточно места для хранения подобной таблицы;
- б) любому проектировщику просто не хватит времени для ее создания;
- в) ни один агент не сможет обучиться всему, что содержится во всех правильных записях таблицы на собственном опыте.

Простые рефлексные агенты

Агенты выбирают действия на основе текущего акта восприятия, игнорируя всю остальную историю актов восприятия.

```
function Reflex-Vacuum-Agent([location,status]) returns действие  
action
```

```
if status = Dirty then return Suck  
else if location = A then return Right  
else if location = B then return Left
```

Простые рефлексные агенты



```
function Simple-Reflex-Agent(percept) returns действие action  
static: rules, множество правил условие-действие  
  
state ← Interpret-Input(percept)  
rule ← Rule-Match(state, rules)  
action ← Rule-Action[rule]  
return action
```

Недостаток – агент работает, только если правильное решение может быть принято на основе исключительно текущего восприятия, т.е. если среда является полностью наблюдаемой.

Рефлексные агенты, основанные на модели

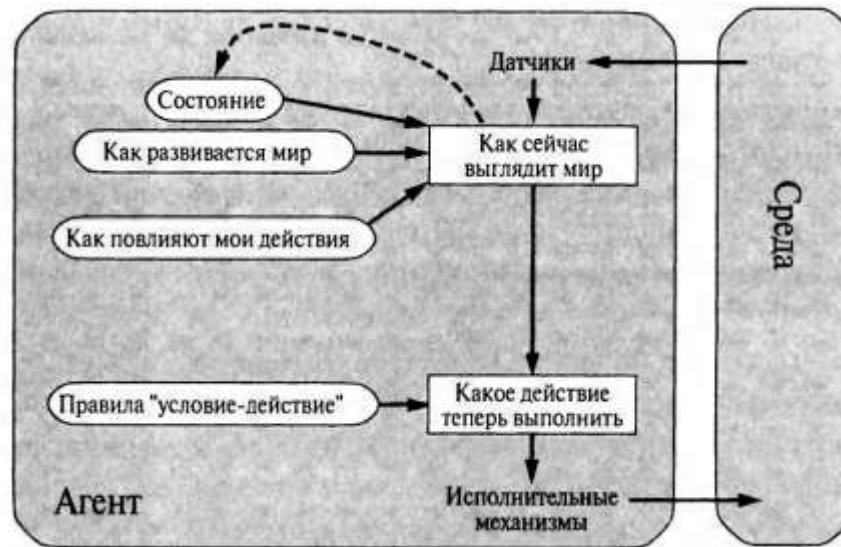
Агент поддерживает внутреннее состояние, которое зависит от истории актов восприятия.

Программа агента содержит знания двух видов:

- информация о том, как мир изменяется независимо от агента (например, что автомобиль, идущий на обгон, обычно становится ближе, чем в какой-то предыдущий момент).
- информация о том, как влияют на мир собственные действия агента (например, при повороте агентом рулевого колеса по часовой стрелке автомобиль поворачивает вправо или что после проезда по автомагистрали в течение пяти минут на север автомобиль находится на пять миль севернее от того места, где он был пять минут назад).

Эти знания о том, «как работает мир» (которые могут быть воплощены в простых логических схемах или в сложных научных теориях) называются **моделью мира**.

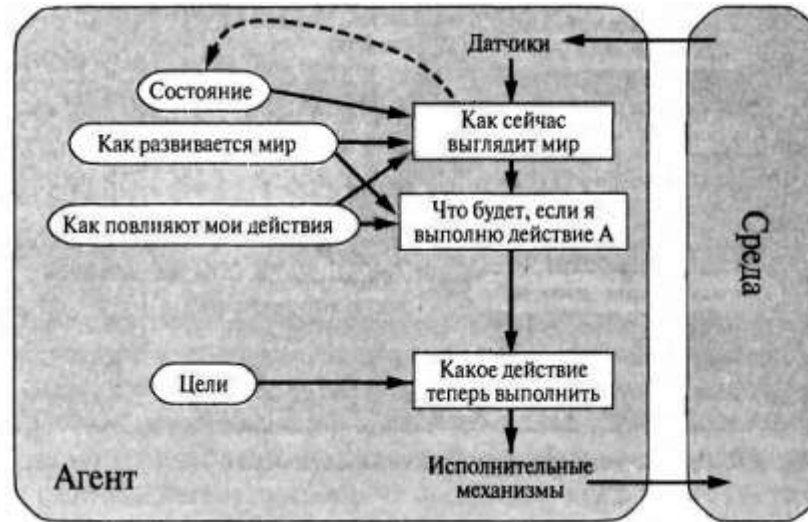
Рефлексные агенты, основанные на модели



```
function Reflex-Agent-With-State(percept) returns действие action
static: state, описание текущего состояния мира
         rules, множество правил условие-действие
         action, последнее по времени действие;
         первоначально не определено
```

```
state ← Update-State(state, action, percept)
rule ← Rule-Match(state, rules)
action ← Rule-Action[rule]
return action
```

***Агенты, основанные на цели

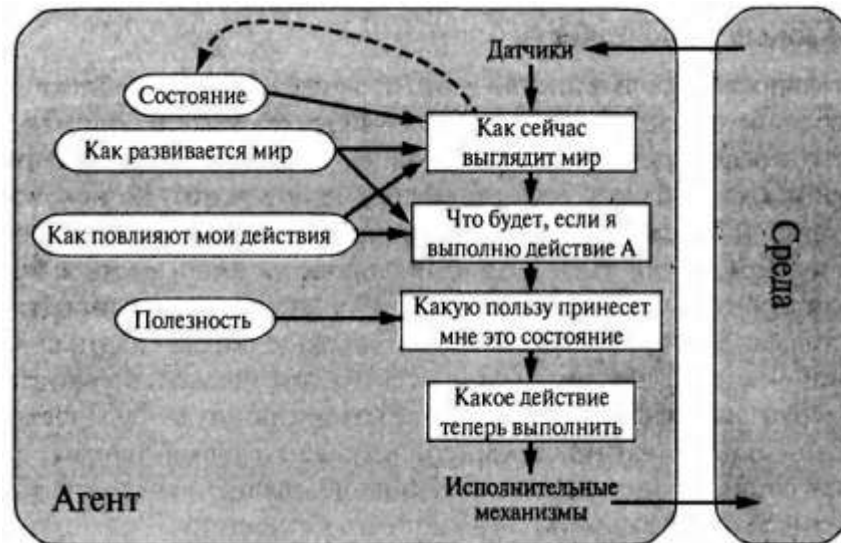


Changing state	Goal-Based Agents	Reflex agents
It starts to rain	update its knowledge of how effectively its brakes will operate; this will automatically cause all of the relevant behaviors to be altered to suit the new conditions.	we would have to rewrite many condition–action rules.
New destination	specify that destination as the goal	The reflex agent’s rules for when to turn and when to go straight will work only for a single destination; they must all be replaced to go somewhere new

Агенты, основанные на полезности

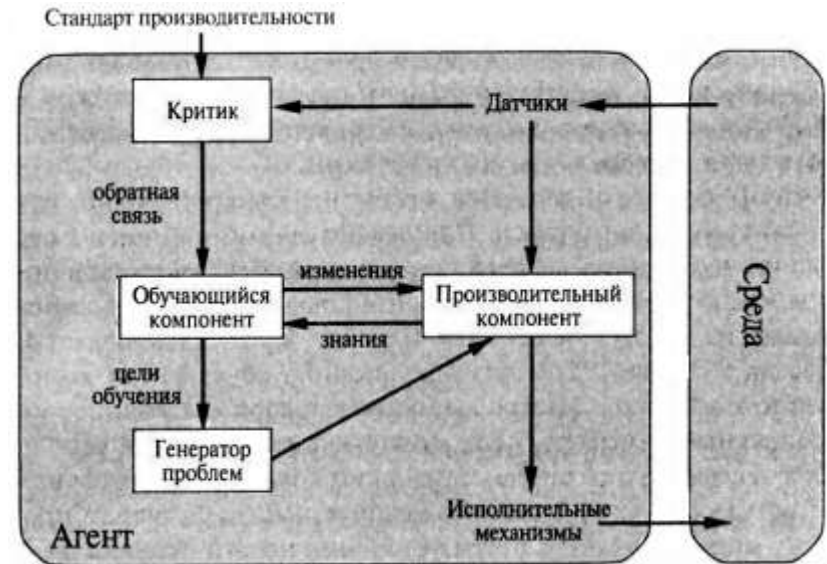
Функция полезности отображает состояние (или последовательность состояний) на вещественное число, которое обозначает соответствующую степень удовлетворенности агента. Полная спецификация функции полезности обеспечивает возможность принимать рациональные решения в следующих двух случаях, когда этого не позволяют сделать цели:

- если имеются конфликтующие цели, такие, что могут быть достигнуты только некоторые из них (например, или скорость, или безопасность), то функция полезности позволяет найти приемлемый компромисс.
- если имеется несколько целей, к которым может стремиться агент, но ни одна из них не может быть достигнута со всей определенностью, то функция полезности предоставляет удобный способ взвешенной оценки вероятности успеха с учетом важности целей.



Обучающиеся агенты

Обучение позволяет агенту функционировать в изначально неизвестных ему вариантах среды и становиться более компетентным по сравнению с тем, что могли бы позволить только его начальные знания.



Основные компоненты:

- обучающийся компонент – отвечает за внесение усовершенствований;
- производительный компонент – обеспечивает выбор внешних действий;
- критик – оценивает, как действует агент, и определяет, каким образом должен быть модифицирован производительный компонент, чтобы он успешнее действовал в будущем;
- генератор проблем – предлагает действия, которые должны привести к получению нового информативного опыта.

Решение проблем посредством поиска

Агенты, решающие задачи

Разновидность агентов, основанных на цели.

Цели позволяют организовать поведение, ограничивая выбор промежуточных этапов, которые пытается осуществить агент. Первым шагом в решении задачи является **формулировка цели** с учетом текущей ситуации и показателей производительности агента.

Цель – множество состояний мира, а именно тех состояний, в которых достигается такая цель. Задача агента состоит в том, чтобы определить, какая последовательность действий приведет его в целевое состояние.

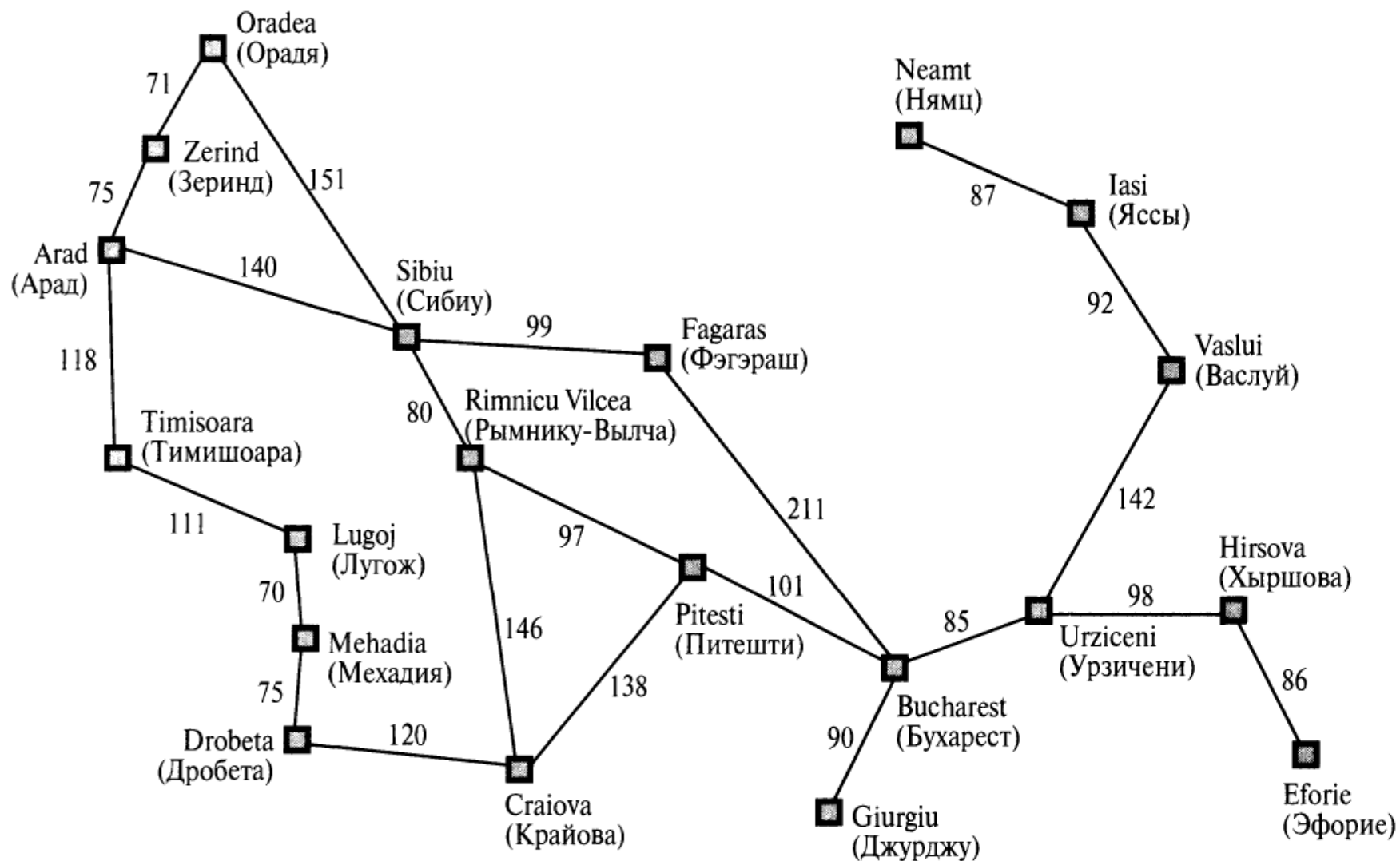
Прежде чем это сделать, агент должен определить, какого рода действия и состояния ему необходимо рассмотреть (степень детализации).

Формулировка задачи представляет собой процесс определения того, какие действия и состояния следует рассматривать с учетом некоторой цели.

Пример. Выходные в Румынии

- Currently in Arad.
- Flight leaves tomorrow from Bucharest
- Formulate goal:
 - be in Bucharest
- Formulate problem:
 - **states**: various cities
 - **actions**: drive between cities
- Find solution:
 - sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

Пример. Выходные в Румынии



Основные определения

Процесс определения последовательности действий, которые приводят к достижению цели, называется *поиском*.

Любой алгоритм поиска принимает в качестве входных данных некоторую задачу и возвращает *решение* в форме последовательности действий.

После того как решение найдено, могут быть осуществлены действия, рекомендованные этим алгоритмом. Такое осуществление происходит на стадии *выполнения*.

```
function Simple-Problem-Solving-Agent(percept) returns действие action  
  inputs: percept, результат восприятия  
  static: seq, последовательность действий, первоначально пустая  
           state, некоторое описание текущего состояния мира  
           goal, цель, первоначально неопределенная  
           problem, формулировка задачи  
  
  state ← Update-State(state, percept)  
  if последовательность seq пуста then do  
    goal ← Formulate-Goal(state)  
    problem ← Formulate-Problem(state, goal)  
    seq ← Search(problem)  
  action ← First(seq)  
  seq ← Rest(seq)  
  return action
```

Хорошо структурированные задачи и решения

Задача может быть формально определена с помощью следующих компонентов:

- **Начальное состояние**, в котором агент приступает к работе.

In(Arad)

- Описание возможных действий, доступных агенту (множество **операторов**, **функция определения преемника**).

$\{ \langle \text{Go}\{\text{Sibiu}\}, \text{In}\{\text{Sibiu}\} \rangle, \langle \text{Go}(\text{Timisoara}), \text{In}(\text{Timisoara}) \rangle, \\ \langle \text{Go}(\text{Zerind}), \text{In}(\text{Zerind}) \rangle \}$

Начальное состояние и функция определения преемника, вместе взятые, неявно задают **пространство состояний** данной задачи — множество всех состояний, достижимых из начального состояния. Пространство состояний образует граф, узлами которого являются состояния, а дугами между узлами — действия.

Путем в пространстве состояний является последовательность состояний, соединенных последовательностью действий.

Хорошо структурированные задачи и решения

- **Проверка цели**, позволяющая определить, является ли данное конкретное состояние целевым состоянием. Иногда имеется явно заданное множество возможных целевых состояний, и эта проверка сводится просто к определению того, является ли данное состояние одним из них.

{In {Bucharest) }.

Иногда цель задается в виде абстрактного свойства, а не явно перечисленного множества состояний. Например, в шахматах цель состоит в достижении состояния, называемого "матом", в котором король противника атакован и не может уклониться от удара.

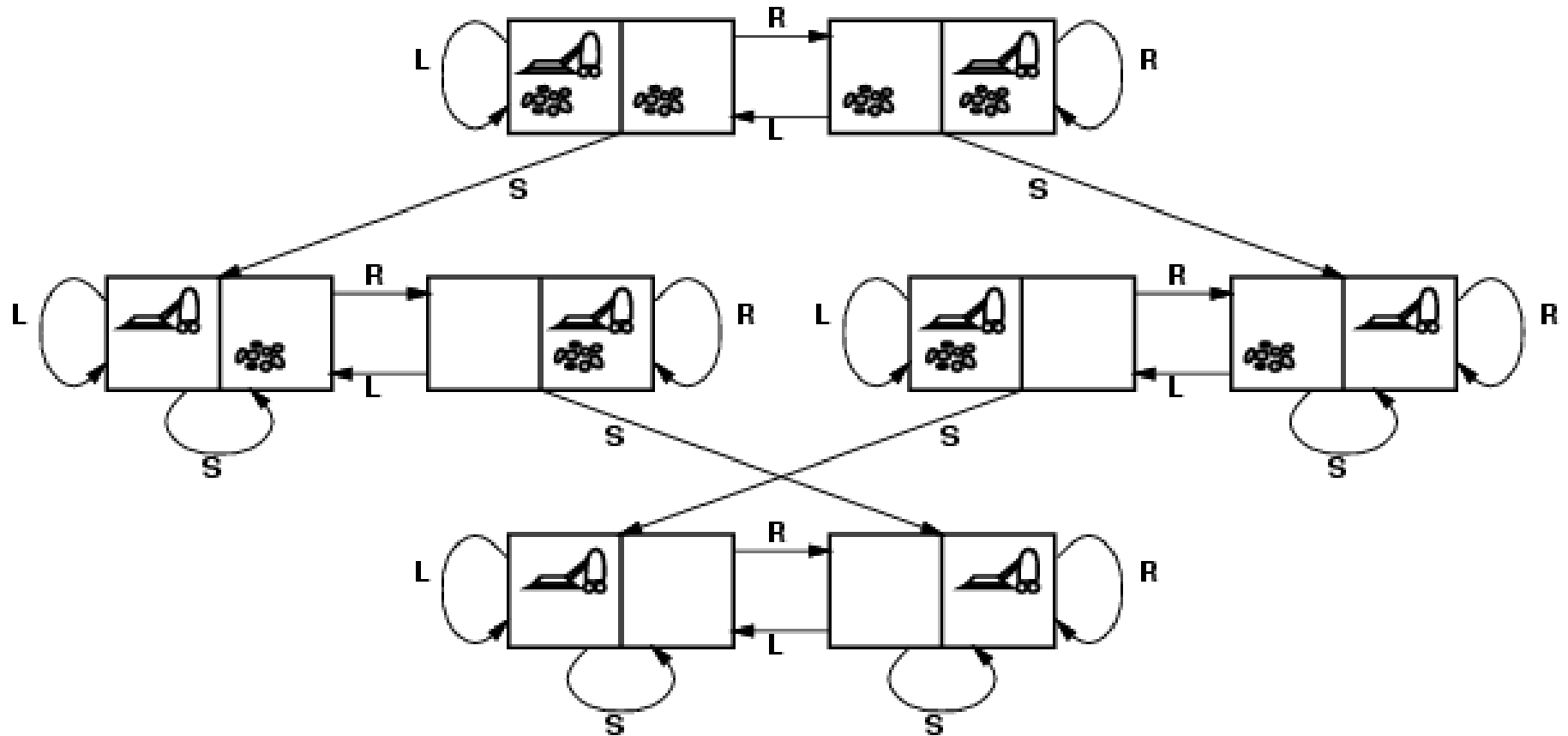
- **Функция стоимости пути**, которая назначает числовое значение стоимости каждому пути. Агент, решающий задачу, выбирает функцию стоимости, которая соответствует его собственным показателям производительности.

$c(x, a, y)$

Решением задачи является путь от начального состояния до целевого состояния.

Оптимальным решением является такое решение, которое имеет наименьшую стоимость пути среди всех прочих решений.

Пространство состояний для мира пылесоса



Мир пылесоса

- **Состояния.** Агент находится в одном из двух местонахождений, в каждом из которых может присутствовать или не присутствовать мусор. Поэтому существует $2 \times 2^2 = 8$ возможных состояний мира.
- **Начальное состояние.** В качестве начального состояния может быть назначено любое состояние.
- **Функция определения преемника.** Эта функция вырабатывает допустимые состояния, которые являются следствием попыток выполнения трех действий (Left, Right и Suck).
- **Проверка цели.** Эта проверка сводится к определению того, являются ли чистыми все квадраты.
- **Стоимость пути.** Стоимость каждого этапа равна 1, поэтому стоимость пути представляет собой количество этапов в этом пути.

Задача игры в восемь

7	2	4
5		6
8	3	1

Начальное состояние

	1	2
3	4	5
6	7	8

Целевое состояние

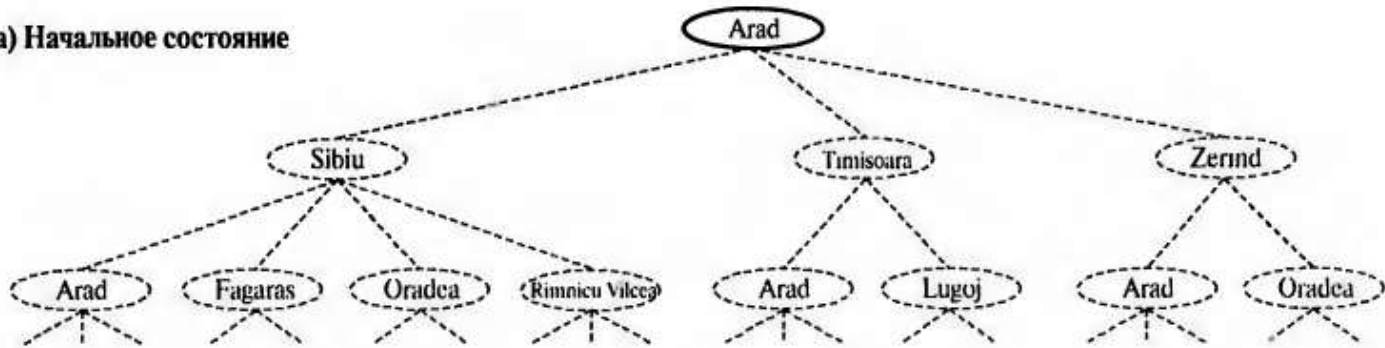
Задача игры в восемь имеет $9!/2 = 181\,440$ достижимых состояний и легко решается.

Задача игры в пятнадцать (на доске 4×4) имеет около 1,3 триллиона состояний, и случайно выбранные ее экземпляры могут быть решены оптимальным образом за несколько миллисекунд с помощью наилучших алгоритмов поиска.

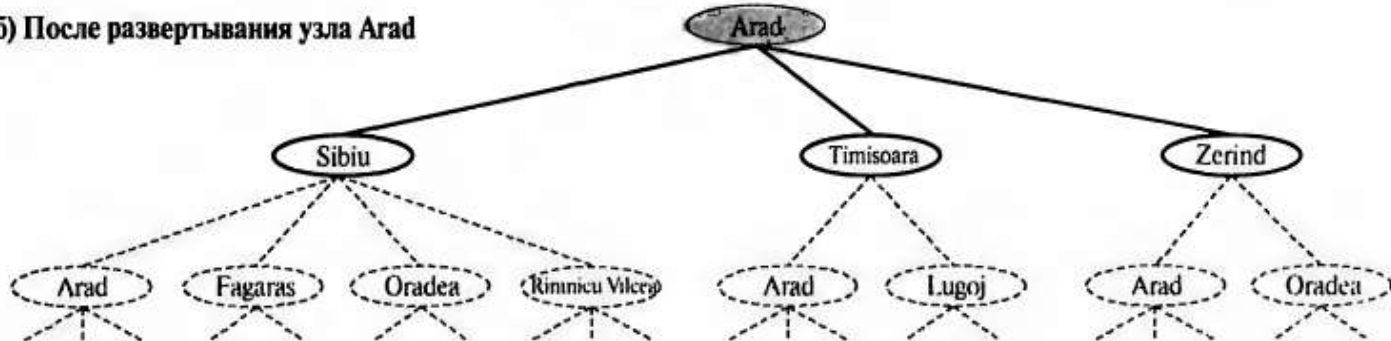
Задача игры в двадцать четыре (на доске 5×5) имеет количество состояний около 10^{25} , и случайно выбранные ее экземпляры все еще весьма нелегко решить оптимальным образом с применением современных компьютеров и алгоритмов.

Поиск решений

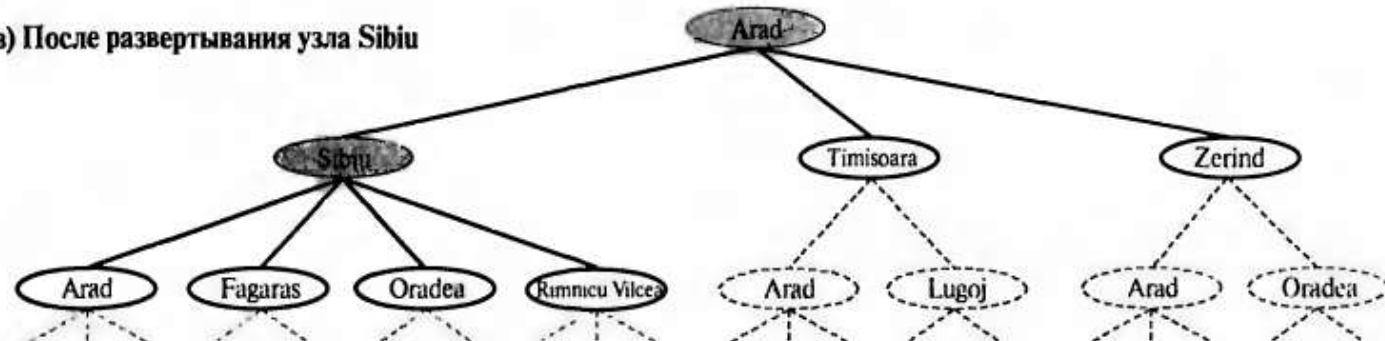
а) Начальное состояние



б) После развертывания узла Arad



в) После развертывания узла Sibiu



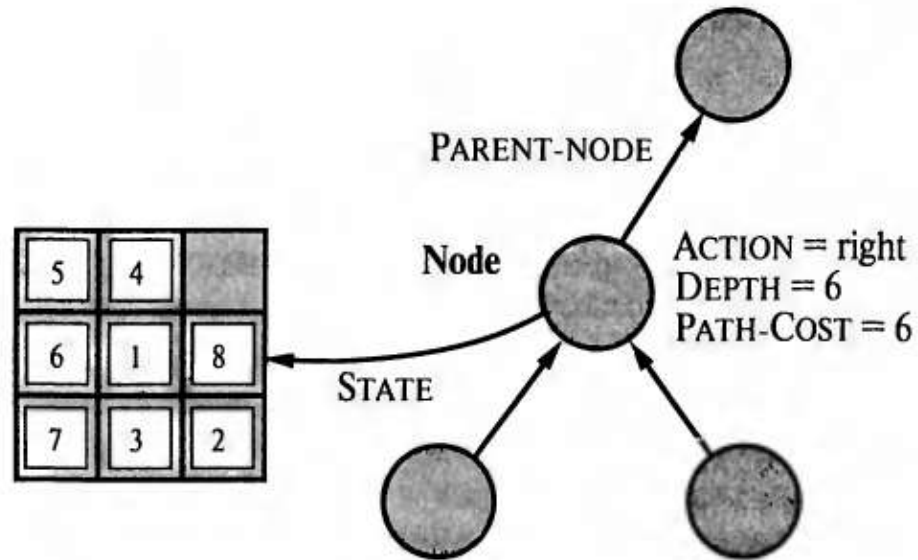
Представление узлов

- *State*. Состояние в пространстве состояний, которому соответствует данный узел.
- *Parent-Node*. Узел в дереве поиска, применявшийся для формирования данного узла (родительский узел).
- *Action*. Действие, которое было применено к родительскому узлу для формирования данного узла.
- *Path-Cost*. Стоимость пути (от начального состояния до данного узла), показанного с помощью указателей родительских узлов, которую принято обозначать как $g(n)$.
- *Depth*. Количество этапов пути от начального состояния, называемое также глубиной.

Необходимо различать узлы и состояния. *Узел* — это учетная структура данных, применяемая для представления дерева поиска, а *состояние* соответствует конфигурации мира. Поэтому узлы лежат на конкретных путях, которые определены с помощью указателей Parent-Node, а состояния — нет.

Два разных узла могут включать одно и то же состояние мира, если это состояние формируется с помощью двух различных путей поиска.

Представление узлов



Коллекция узлов, которые были сформированы, но еще не развернуты, называется *периферией*. Каждый элемент периферии представляет собой *листовой узел*, т.е. узел, не имеющий преемников в дереве.

Коллекция узлов м.б. выражена в виде очереди.

Измерение производительности решения задачи

- **Полнота.** Гарантирует ли алгоритм обнаружение решения, если оно имеется?
- **Оптимальность.** Обеспечивает ли данная стратегия нахождение оптимального решения.
- **Временная сложность.** За какое время алгоритм находит решение?
- **Пространственная сложность.** Какой объем памяти необходим для осуществления поиска?

В искусственном интеллекте, где граф представлен неявно с помощью начального состояния и функции определения преемника и часто является бесконечным, сложность выражается в терминах трех величин: b – коэффициент ветвления или максимальное количество преемников любого узла, d – глубина самого поверхностного целевого узла и m – максимальная длина любого пути в пространстве состояний.

Стратегии неинформированного поиска

Uninformed search strategies use only the information available in the problem definition.

- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening search

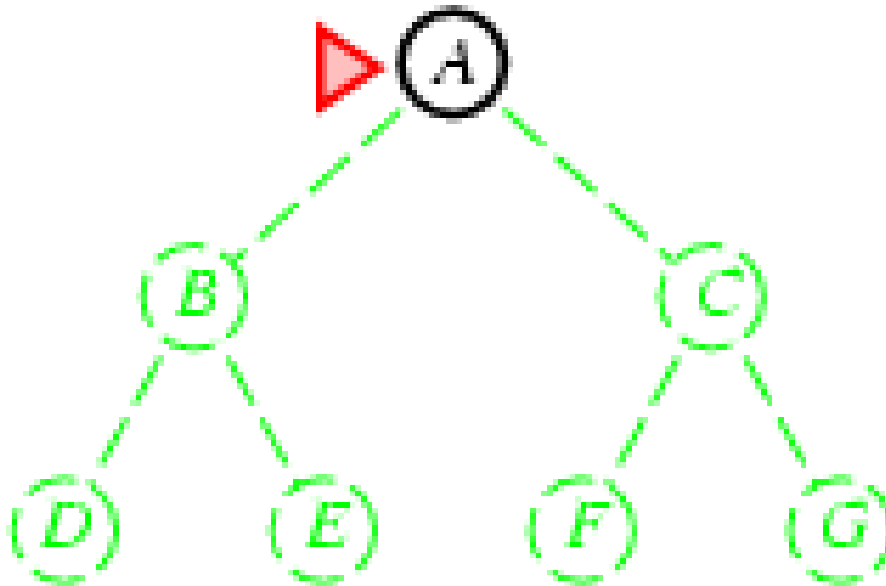
Uninformed search strategies

- A search strategy is defined by picking the **order of node expansion**
- Strategies are evaluated along the following dimensions:
 - **completeness**: Is the algorithm guaranteed to find a solution when there is one?
 - **optimality**: Does the strategy find the optimal solution?
 - **time complexity**: How long does it take to find a solution?
 - **space complexity**: How much memory is needed to perform the search?
- Time and space complexity are measured in terms of
 - b : the branching factor or maximum number of successors of any node
 - d : the depth of the shallowest goal node
 - m : the maximum length of any path in the state space

Breadth-first search

Expand shallowest unexpanded node

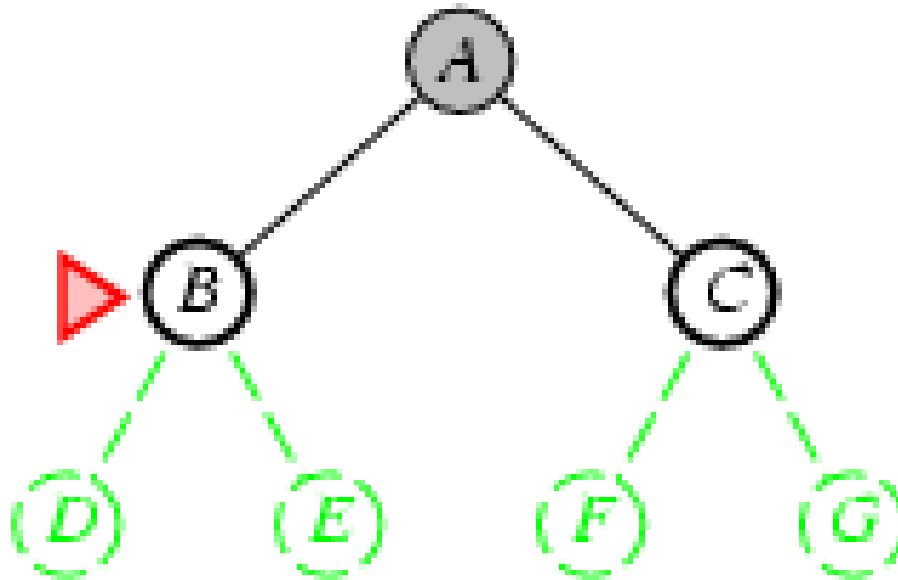
Frontier is a FIFO queue, i.e., new successors go at end



Breadth-first search

Expand shallowest unexpanded node

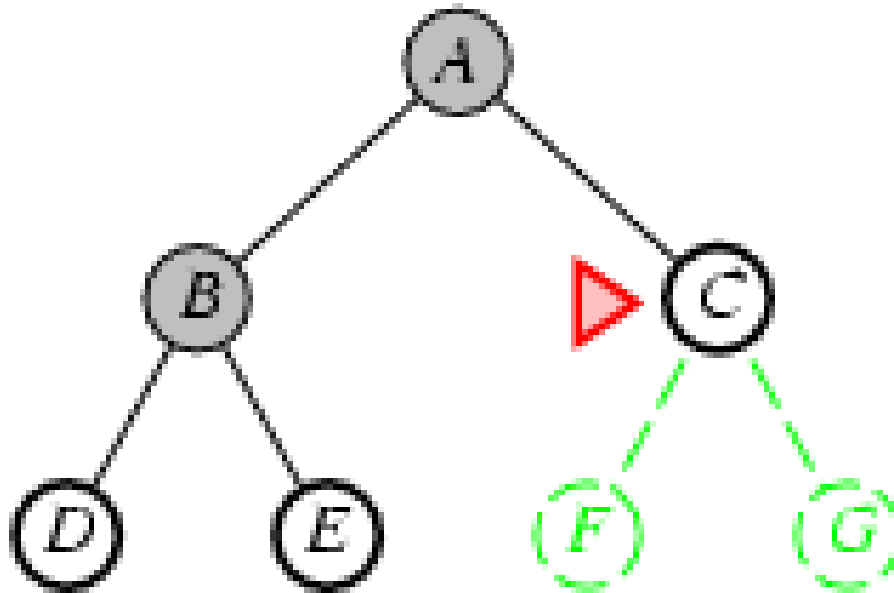
Frontier is a FIFO queue, i.e., new successors go at end



Breadth-first search

Expand shallowest unexpanded node

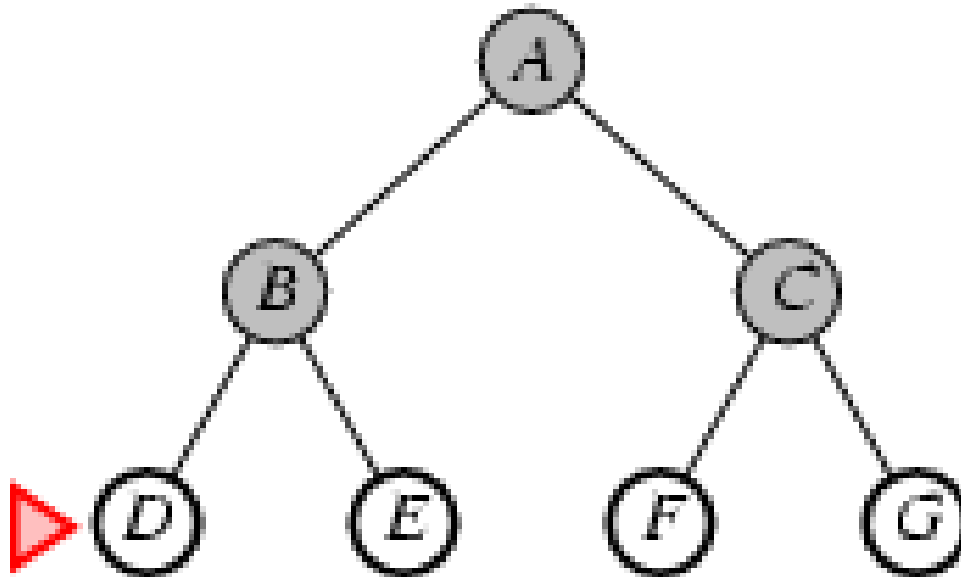
Frontier is a FIFO queue, i.e., new successors go at end



Breadth-first search

Expand shallowest unexpanded node

Frontier is a FIFO queue, i.e., new successors go at end



Breadth-first search

function BREADTH-FIRST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

frontier $\leftarrow$ a FIFO queue with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the shallowest node in *frontier* */

 add *node*.STATE to *explored*

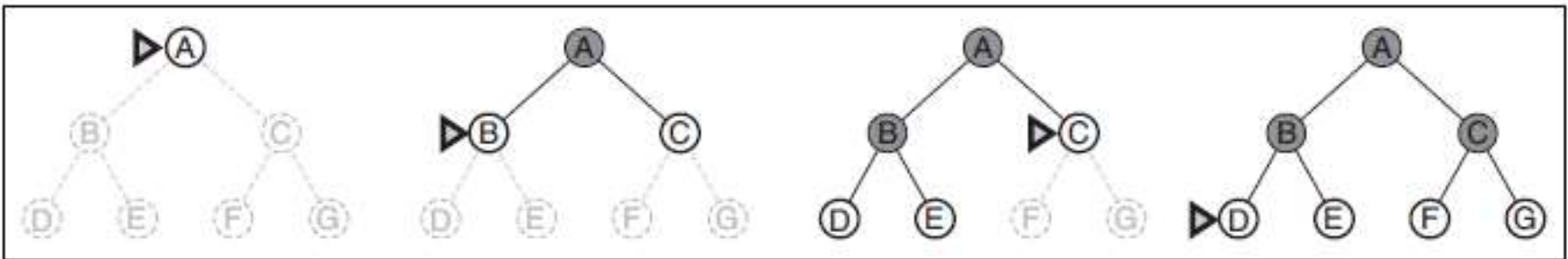
for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

if *problem*.GOAL-TEST(*child*.STATE) **then return** SOLUTION(*child*)

frontier $\leftarrow$ INSERT(*child*, *frontier*)



Properties of breadth-first search

- Complete? Yes (if b is finite)
- Time? $1+b+b^2+b^3+\dots +b^d + b(b^d-1) = O(b^{d+1})$
- Space? $O(b^{d+1})$ (keeps every node in memory)
- Optimal? Yes (if cost = 1 per step)

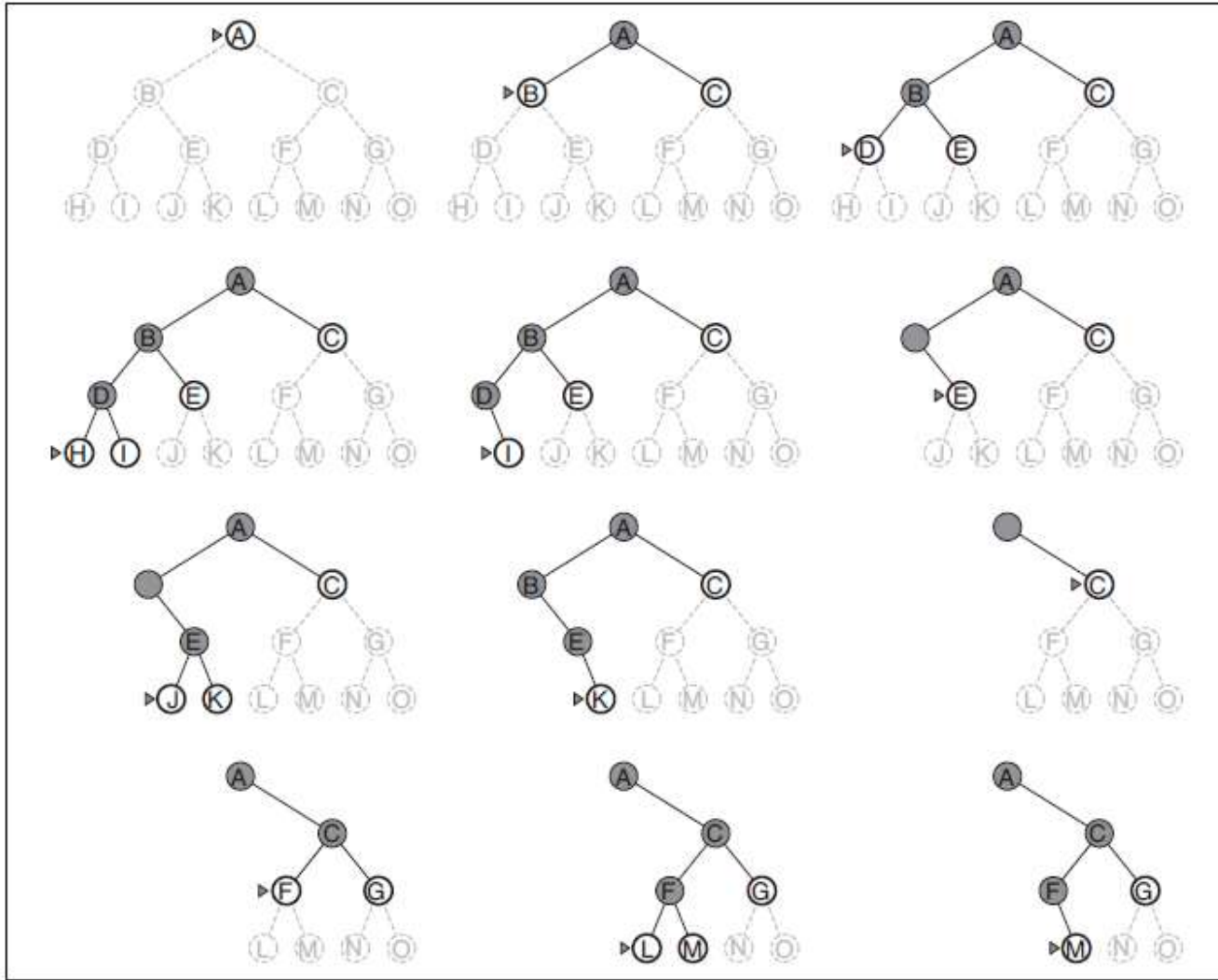
- **Space** is the bigger problem (more than time)

Uniform-cost search

- Expand least-cost unexpanded node
- Frontier is a queue ordered by path cost
- Equivalent to breadth-first if step costs all equal
- Complete? Yes, if step cost $\geq \epsilon$
- Time? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$ where C^* is the cost of the optimal solution
- Space? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$
- Optimal? Yes – nodes expanded in increasing order of $g(n)$

Depth-first search

Expand deepest unexpanded node. Frontier = LIFO queue, i.e., put successors at front



***Properties of depth-first search

- Complete? No: fails in infinite-depth spaces, spaces with loops
 - Modify to avoid repeated states along path
→ complete in finite spaces
- Time? $O(b^m)$: terrible if m is much larger than d
 - but if solutions are dense, may be much faster than breadth-first
- Space? $O(bm)$, i.e., linear space!
- Optimal? No

Depth-limited search

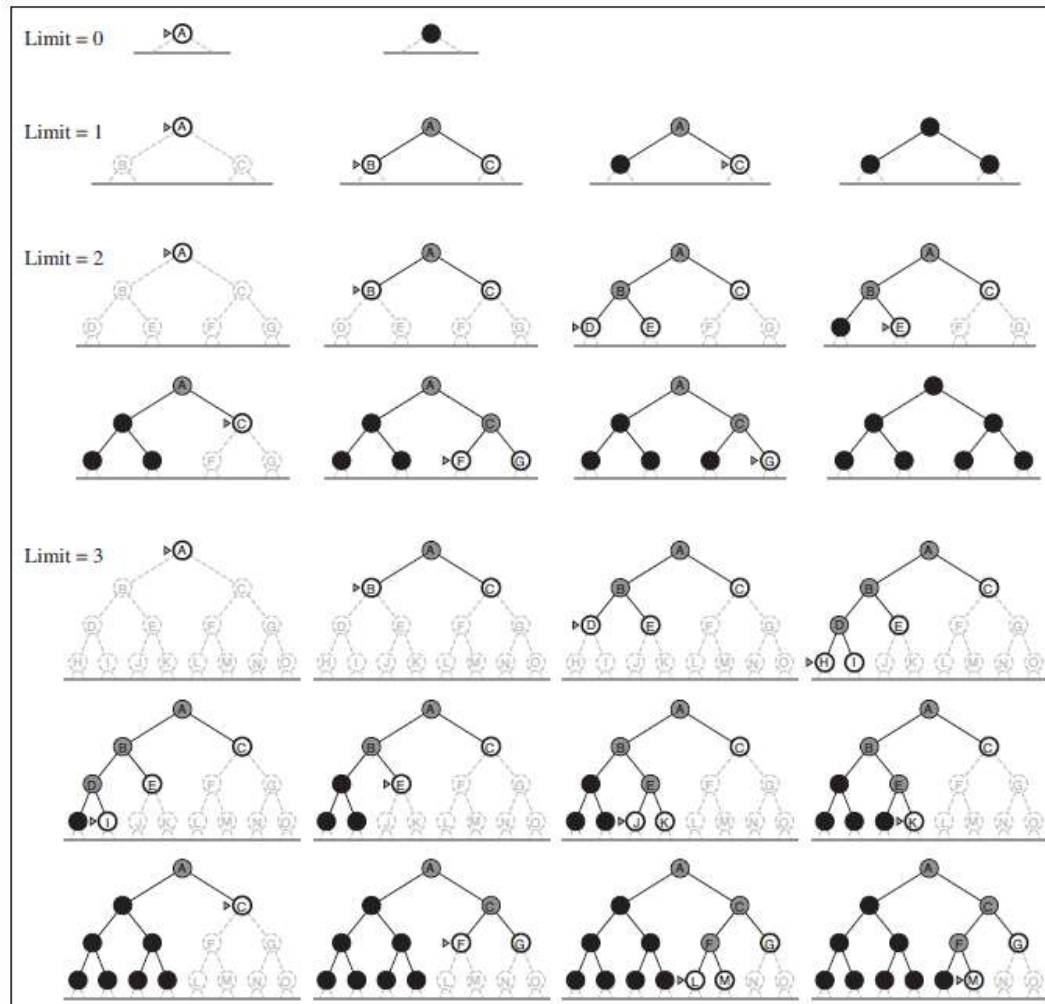
= depth-first search with depth limit l ,
i.e., nodes at depth l have no successors

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff  
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)  
  
function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff  
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)  
  else if limit = 0 then return cutoff  
  else  
    cutoff_occurred?  $\leftarrow$  false  
    for each action in problem.ACTIONS(node.STATE) do  
      child  $\leftarrow$  CHILD-NODE(problem, node, action)  
      result  $\leftarrow$  RECURSIVE-DLS(child, problem, limit - 1)  
      if result = cutoff then cutoff_occurred?  $\leftarrow$  true  
      else if result  $\neq$  failure then return result  
  if cutoff_occurred? then return cutoff else return failure
```

Iterative deepening search

```

function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
  for depth = 0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result
  
```



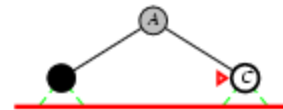
Iterative deepening search

Limit = 0



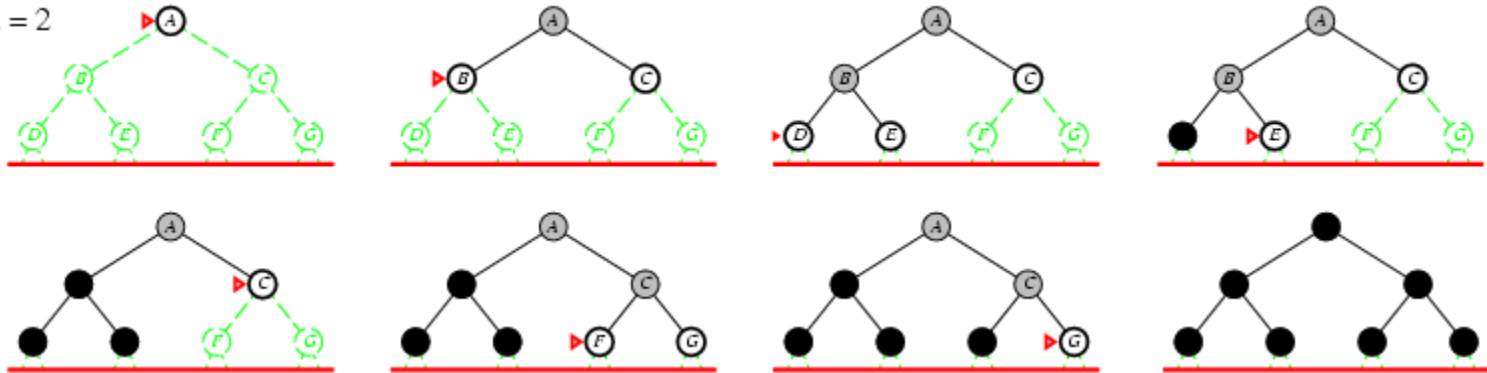
Iterative deepening search

Limit = 1



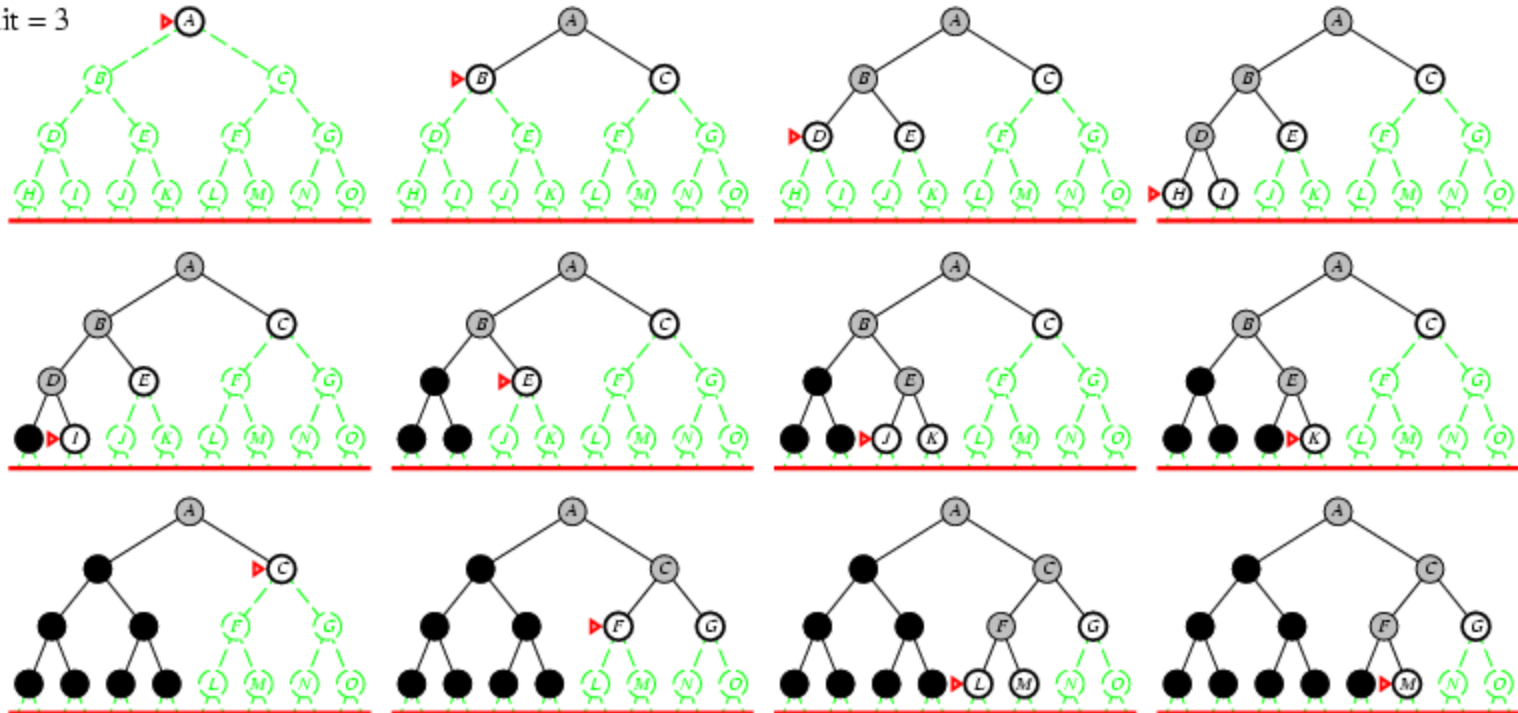
Iterative deepening search

Limit = 2



Iterative deepening search

Limit = 3



Properties of iterative deepening search

- Complete? Yes
- Time? $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step cost = 1

Comparing uninformed search strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: ^a complete if b is finite; ^b complete if step costs $\geq \epsilon$ for positive ϵ ; ^c optimal if step costs are all identical; ^d if both directions use breadth-first search.

Стратегии информированного (эвристического) поиска

Помимо определения задачи используются знания, относящиеся к данной конкретной проблемной области.

Подход носит название *поиск по первому наилучшему совпадению* (best first search).

Ключевой компонент – эвристическая функция $h(n)$ (heuristic) – оценка стоимости наименее дорогостоящего пути от узла n до целевого узла.

Если n – целевой узел, то $h(n)=0$

Алгоритмы:

- Жадный поиск по первому наилучшему совпадению (greedy best-first search)
- Поиск A^*

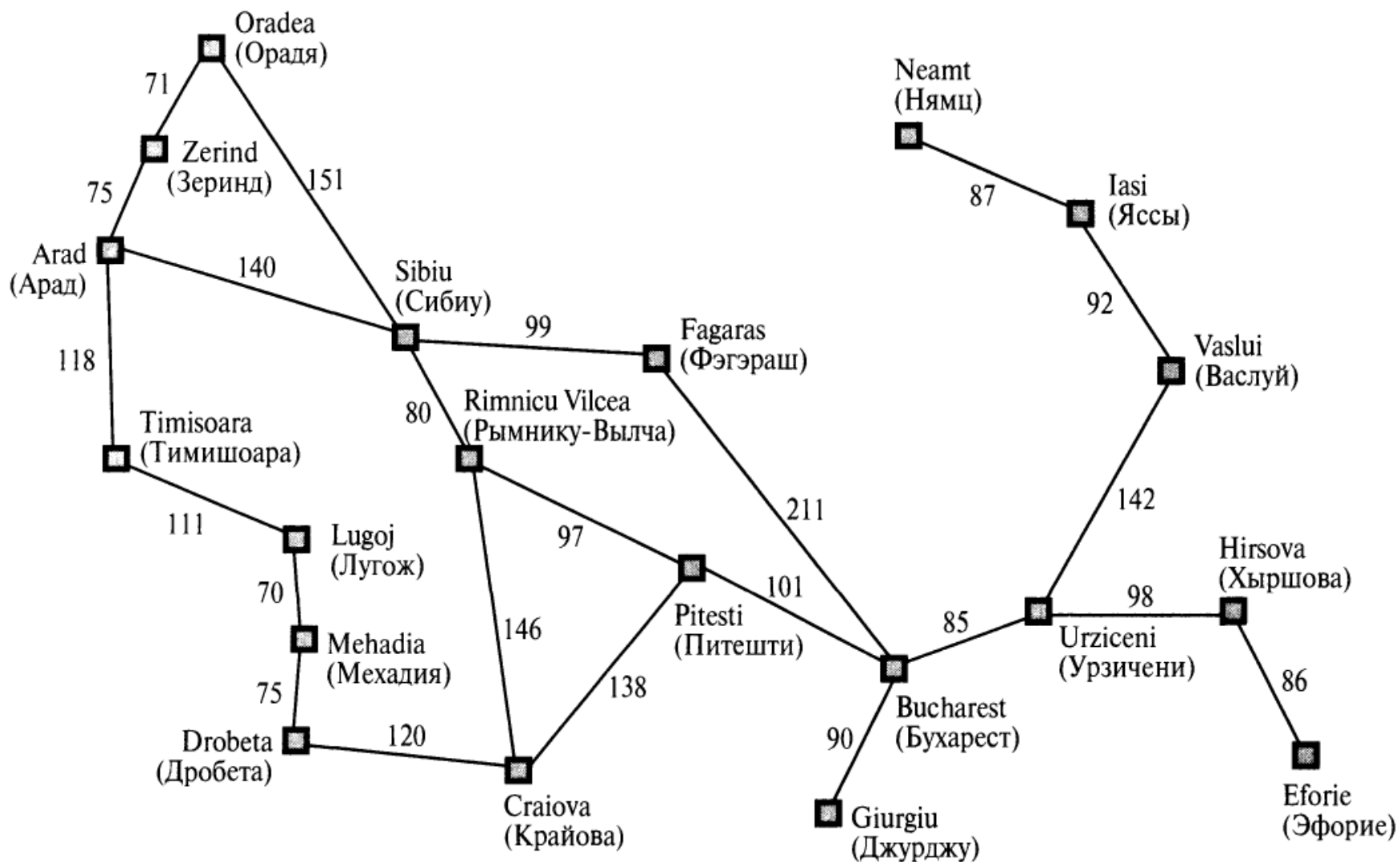
Жадный поиск по первому наилучшему совпадению

Предпринимаются попытки развертывания узла, который рассматривается как ближайший к цели на том основании, что он со всей вероятностью должен быстро привести к решению. Таким образом, при этом поиске оценка узлов производится с использованием только эвристической функции: $f(n)=h(n)$.

Решение задачи поиска маршрута в Румынии на основе эвристической функции определения **расстояния по прямой (Straight Line Distance — SLD)**, для которой принято обозначение h_{SLD} .

Обозначение узла	Название города	Расстояние по прямой до Бухареста	Обозначение узла	Название города	Расстояние по прямой до Бухареста
<i>Arad</i>	Арад	366	<i>Mehadia</i>	Мехадия	241
<i>Bucharest</i>	Бухарест	0	<i>Neamt</i>	Нямц	234
<i>Craiova</i>	Крайова	160	<i>Oradea</i>	Орадя	380
<i>Drobeta</i>	Дробета	242	<i>Pitesti</i>	Питешти	100
<i>Eforie</i>	Эфорие	161	<i>RimnicuVilcea</i>	Рымнику-Вылча	193
<i>Fagaras</i>	Фэгэраш	176	<i>Sibiu</i>	Сибиу	253
<i>Giurgiu</i>	Джурджу	77	<i>Timisoara</i>	Тимишоара	329
<i>Hirsova</i>	Хыршова	151	<i>Urziceni</i>	Урзичени	80
<i>Iasi</i>	Яссы	226	<i>Vaslui</i>	Васлуй	199
<i>Lugoj</i>	Лугож	244	<i>Zerind</i>	Зеринд	374

Пример. Выходные в Румынии



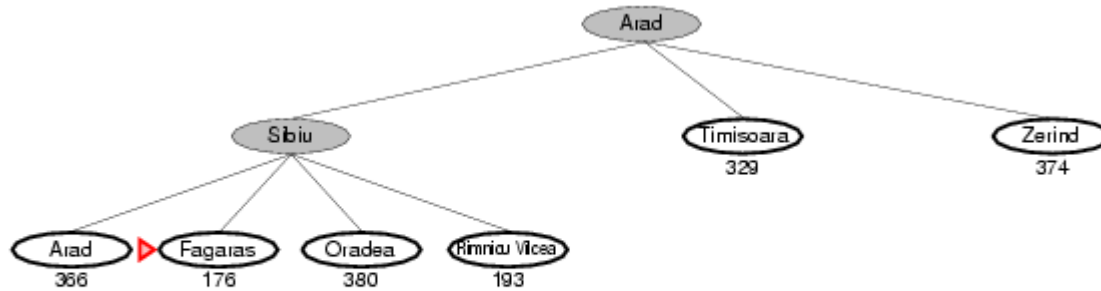
Этапы жадного поиска пути до Бухареста



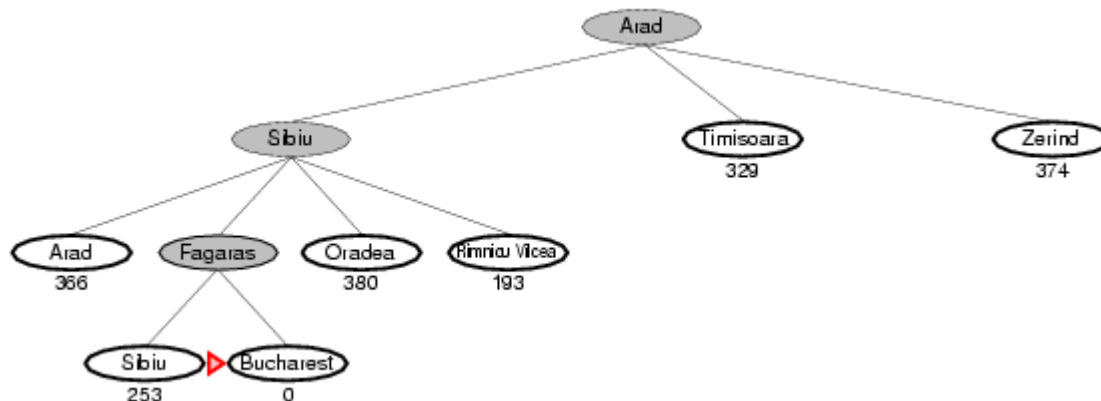
Этапы жадного поиска пути до Бухареста



Этапы жадного поиска пути до Бухареста



Этапы жадного поиска пути до Бухареста



Оптимален ли путь?

Найденное решение не оптимально: путь до Бухареста через города Сибиу и Фэгэраш на 32 километра длиннее, чем путь через города Рымнику-Вылча и Питешти.

Свойства жадного поиска по наилучшему совпадению

- Complete? No – can get stuck in loops, e.g., lasi → Neamt → lasi → Neamt →
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? No

Поиск A*

$$f(n) = g(n) + h(n)$$

$f(n)$ – оценка стоимости наименее дорогостоящего пути решения, проходящего через узел n .

Таким образом, при осуществлении попытки найти наименее дорогостоящее решение, по-видимому, разумнее всего вначале попытаться проверить узел с наименьшим значением $g(n) + h(n)$.

Как оказалось, данная стратегия является больше чем просто разумной: если эвристическая функция $h(n)$ удовлетворяет некоторым условиям, то поиск A* становится и полным, и оптимальным.

Условия оптимальности: допустимость и непротиворечивость

An admissible heuristic is one that never overestimates the cost to reach the goal. Because $g(n)$ is the actual cost to reach n along the current path, and $f(n)=g(n) + h(n)$, we have as an immediate consequence that $f(n)$ never overestimates the true cost of a solution along the current path through n .

Admissible heuristics are by nature optimistic because they think the cost of solving the problem is less than it actually is. An obvious example of an admissible heuristic is the straight-line distance h_{SLD} that we used in getting to Bucharest. Straight-line distance is admissible because the shortest path between any two points is a straight line, so the straight line cannot be an overestimate.

Поиск A^*

Поиск A^* является оптимальным, при условии, что $h(n)$ представляет собой допустимую эвристическую функцию, т.е. при условии, что $h(n)$ никогда не переоценивает стоимость достижения цели.

Допустимые эвристические функции являются по своей сути оптимистическими функциями, поскольку возвращают значения стоимости решения задачи, меньшие по сравнению с фактическими значениями стоимости.

А поскольку $g(n)$ — точная стоимость достижения узла n , из этого непосредственно следует, что функция $f(n)$ никогда не переоценивает истинную стоимость достижения решения через узел n .

Consistent (непротиворечивые) heuristics

- A heuristic is **consistent** if for every node n , every successor n' of n generated by any action a , the estimated cost of reaching the goal from n is no greater than the step cost of getting to n plus the estimated cost of reaching the goal from n :

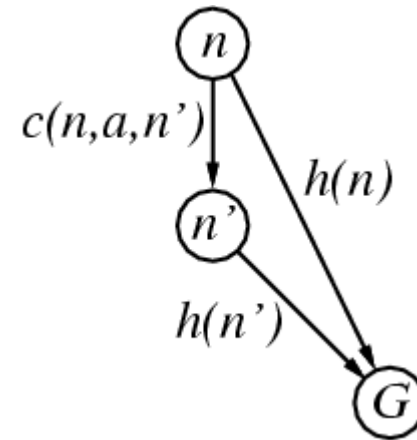
$$h(n) \leq c(n,a,n') + h(n')$$

- If h is consistent, we have

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n,a,n') + h(n') \\ &\geq g(n) + h(n) \\ &= f(n) \end{aligned}$$

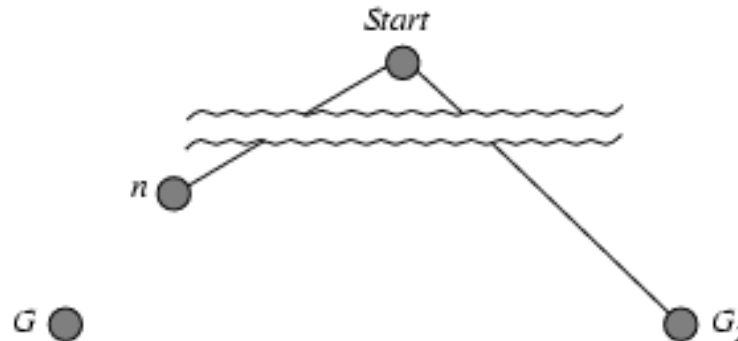
- i.e., $f(n)$ is non-decreasing along any path.

This is a form of the general triangle inequality, which stipulates that each side of a triangle cannot be longer than the sum of the other two sides.



Optimality of A* (proof)

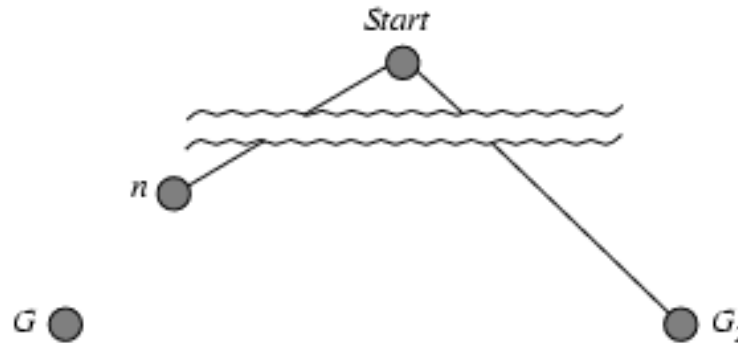
- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let n be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal G .



- $f(G_2) = g(G_2)$ since $h(G_2) = 0$
- $g(G_2) > g(G)$ since G_2 is suboptimal
- $f(G) = g(G)$ since $h(G) = 0$
- $f(G_2) > f(G)$ from above

Optimality of A* (proof)

- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let n be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal G .



- $f(G_2) > f(G)$ from above
- $h(n) \leq h^*(n)$ since h is admissible
- $g(n) + h(n) \leq g(n) + h^*(n)$
- $f(n) \leq f(G)$

Hence $f(G_2) > f(n)$, and A^* will never select G_2 for expansion

Поиск A*

Предположим, что на периферии поиска появился неоптимальный целевой узел G2, а стоимость оптимального решения равна C^* .

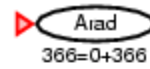
В таком случае, поскольку узел G2 неоптимален, а $h(G2) = 0$ (это выражение справедливо для любого целевого узла), можно вывести следующую формулу:
$$f(G2) = g(G2) + h(G2) = g(G2) > C^*$$

Теперь рассмотрим периферийный узел n , который находится в оптимальном пути решения (если решение существует, то всегда должен быть такой узел).

Если функция $h(n)$ не переоценивает стоимость завершения этого пути решения, то
$$f(n) = g(n) + h(n) \leq C^*$$

$f(n) \leq C^* < f(G2) \rightarrow$ узел G2 не развертывается и поиск A* должен вернуть оптимальное решение.

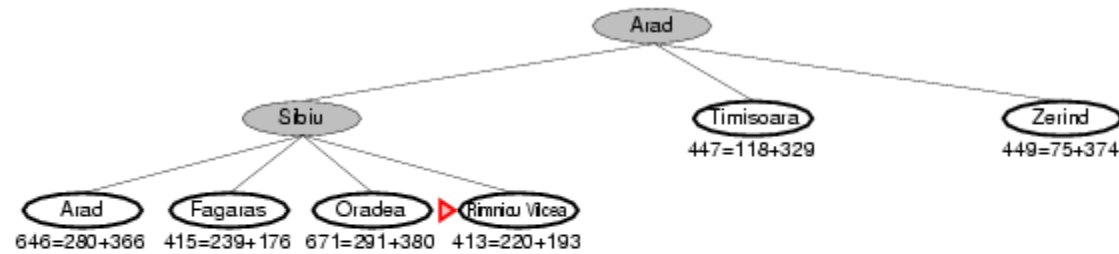
Поиск A*



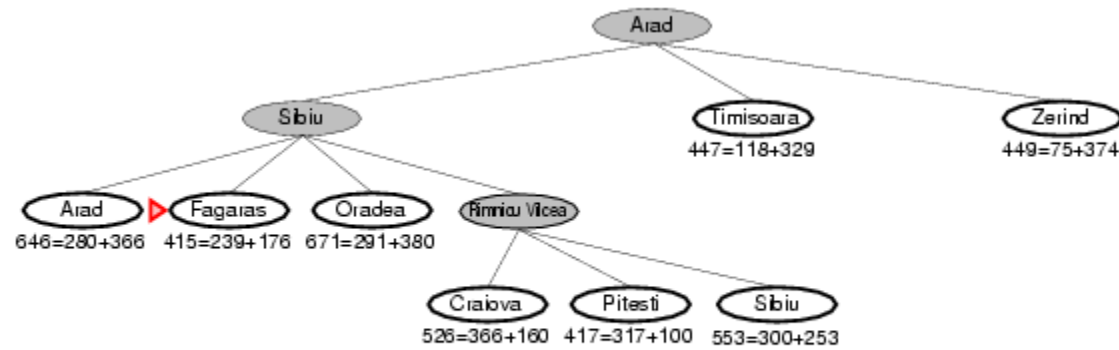
Поиск A*



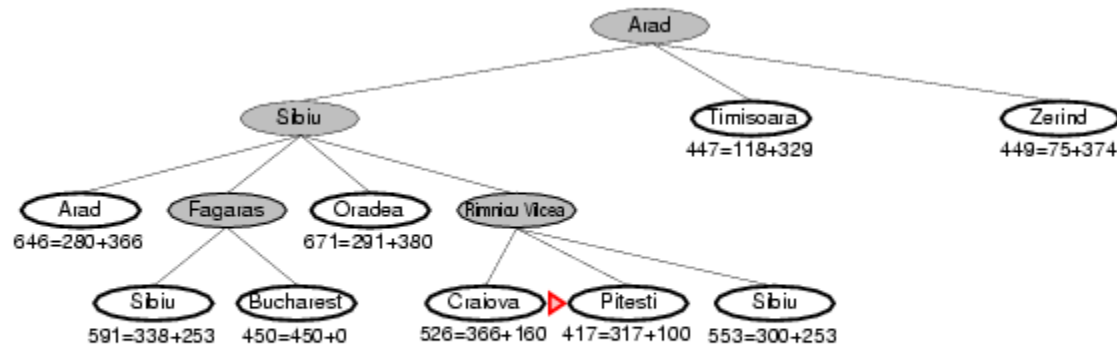
Поиск A*



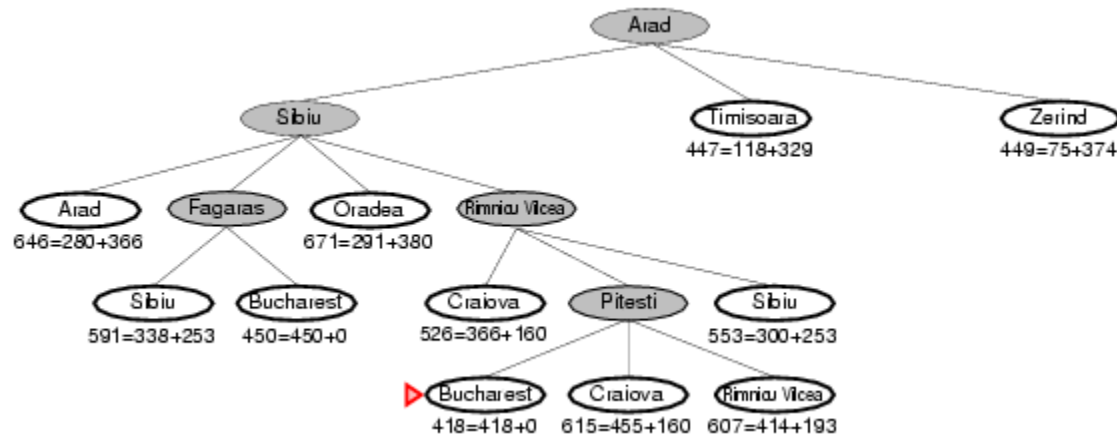
Поиск A*



Поиск A*



Поиск A*



Properties of A*

- Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)
- Time? Exponential
- Space? Keeps all nodes in memory
- Optimal? Yes

Поиск A^*

Если C^* представляет собой стоимость оптимального пути решения, то:

- в поиске A^* развертываются все узлы со значениями $f(n) < C^*$;
- в поиске A^* могут развертываться некоторые дополнительные узлы, находящиеся непосредственно на "целевом контуре" (где $f(n) = C^*$), прежде чем будет выбран целевой узел.

Эвристические функции

- $h_1(n)$ = количество фишек, стоящих не на своем месте.
- $h_2(n)$ = сумма расстояний всех фишек от их целевых позиций (манхэттенское расстояние).

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(S) = ?$
- $h_2(S) = ?$

Эвристические функции

- $h_1(n)$ = количество фишек, стоящих не на своем месте.
- $h_2(n)$ = сумма расстояний всех фишек от их целевых позиций (манхэттенское расстояние).

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(S) = ?$ 8
- $h_2(S) = ?$ $3+1+2+2+2+3+3+2 = 18$

Решение имеет длину 26 ходов.

Сравнение алгоритмов поиска

d	Стоимость поиска			Эффективный коэффициент ветвления		
	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2,45	1,79	1,79
4	112	13	12	2,87	1,48	1,45
6	680	20	18	2,73	1,34	1,30
8	6384	39	25	2,80	1,33	1,24
10	47127	93	39	2,79	1,38	1,22
12	3644035	227	73	2,78	1,42	1,24
14	—	539	113	—	1,44	1,23
16	—	1301	211	—	1,45	1,25
18	—	3056	363	—	1,46	1,26
20	—	7276	676	—	1,47	1,27
22	—	18094	1219	—	1,48	1,28
24	—	39135	1641	—	1,48	1,26

Доминирование

- If $h_2(n) \geq h_1(n)$ for all n (both admissible)
- then h_2 **dominates** h_1
- h_2 is better for search
- Typical search costs (average number of nodes expanded):
 - $d=12$ IDS = 3,644,035 nodes
 $A^*(h_1) = 227$ nodes
 $A^*(h_2) = 73$ nodes
 - $d=24$ IDS = too many nodes
 $A^*(h_1) = 39,135$ nodes
 $A^*(h_2) = 1,641$ nodes

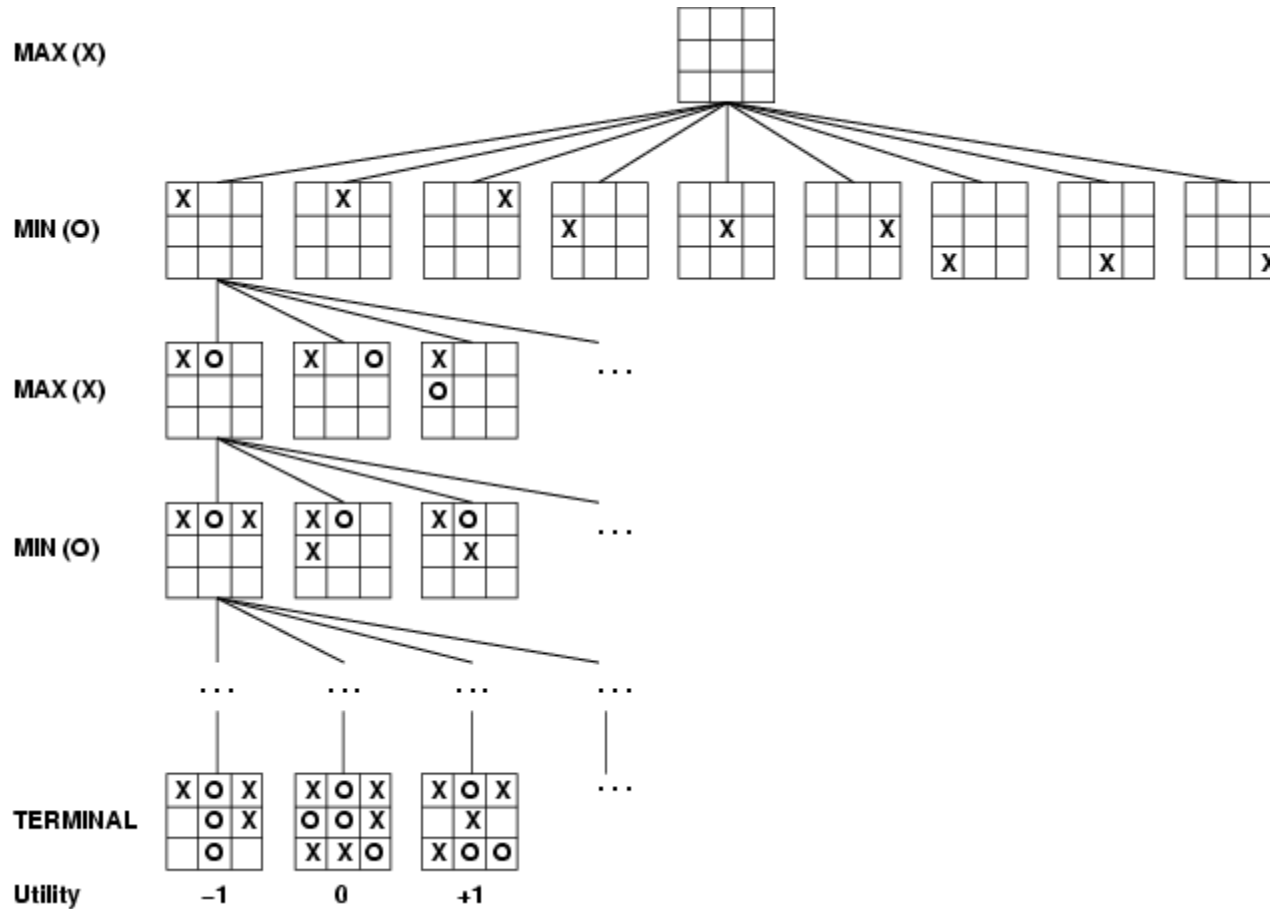
Поиск в условиях противодействия (Игры)

Game definition

A game can be formally defined as a kind of search problem with the following elements:

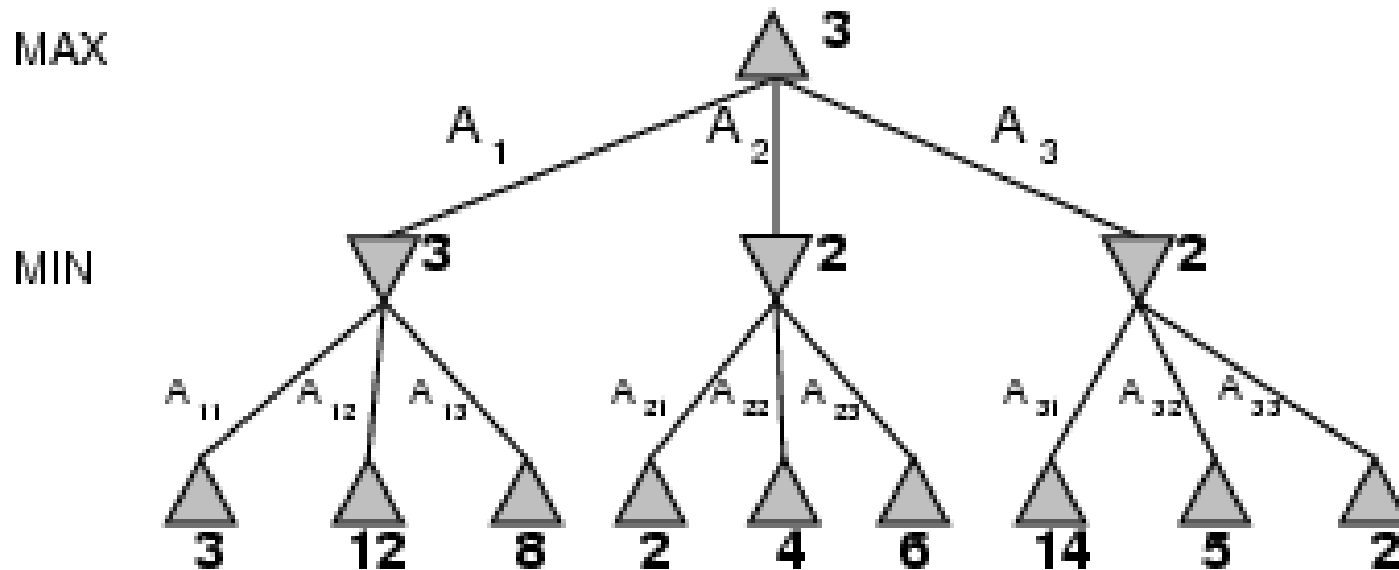
- S_0 : The initial state, which specifies how the game is set up at the start.
- $PLAYER(s)$: Defines which player has the move in a state.
- $ACTIONS(s)$: Returns the set of legal moves in a state.
- $RESULT(s, a)$: The transition model, which defines the result of a move.
- $TERMINAL-TEST(s)$: A terminal test, which is true when the game is over and false otherwise. States where the game has ended are called terminal states.
- $UTILITY(s, p)$: A utility function (also called an objective function or payoff function), defines the final numeric value for a game that ends in terminal state s for a player p . A zero-sum game is defined as one where the total payoff to all players is the same for every instance of the game.

Game tree (2-player, deterministic, turns)



Minimax

- Perfect play for deterministic games
- Idea: choose move to position with highest **minimax value**
= best achievable payoff against best play
- E.g., 2-ply game:



Minimax algorithm

function MINIMAX-DECISION(*state*) *returns an action*

$v \leftarrow \text{MAX-VALUE}(\textit{state})$

return the *action* in SUCCESSORS(*state*) with value *v*

function MAX-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$

return *v*

function MIN-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

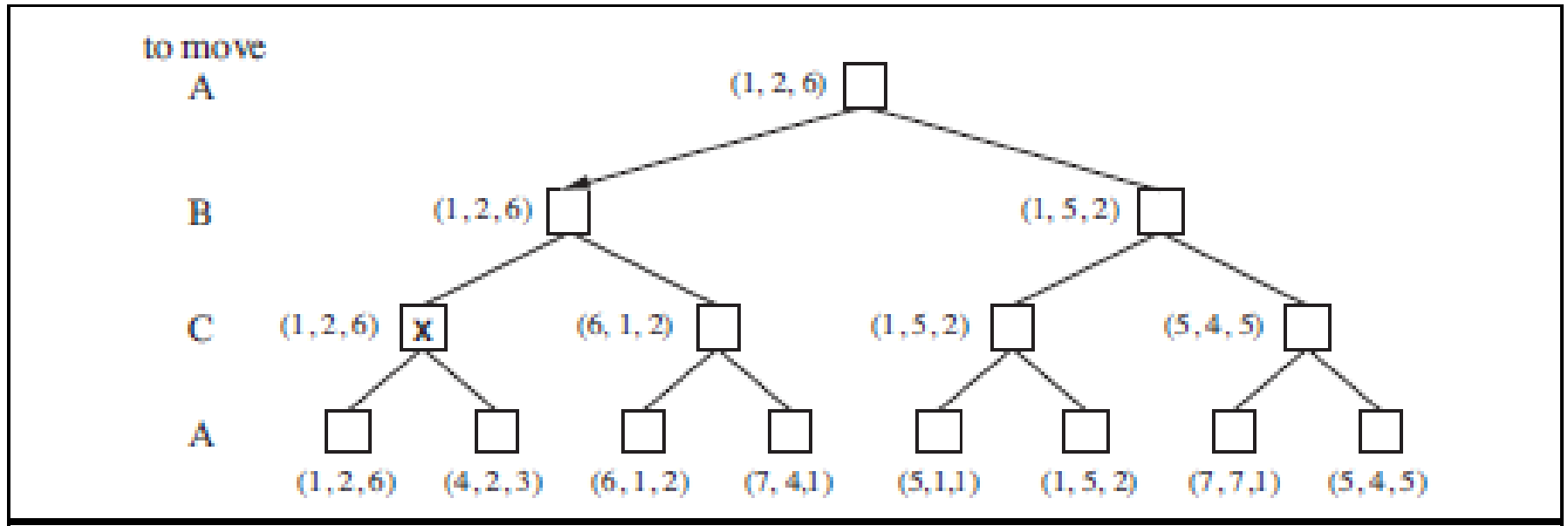
$v \leftarrow \infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$

return *v*

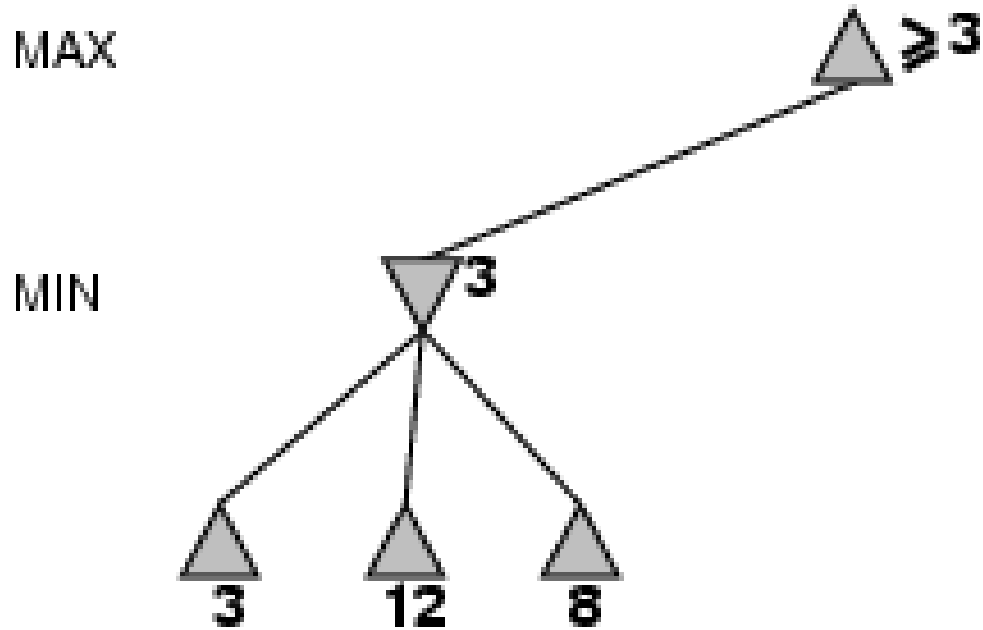
Optimal decisions in multiplayer games



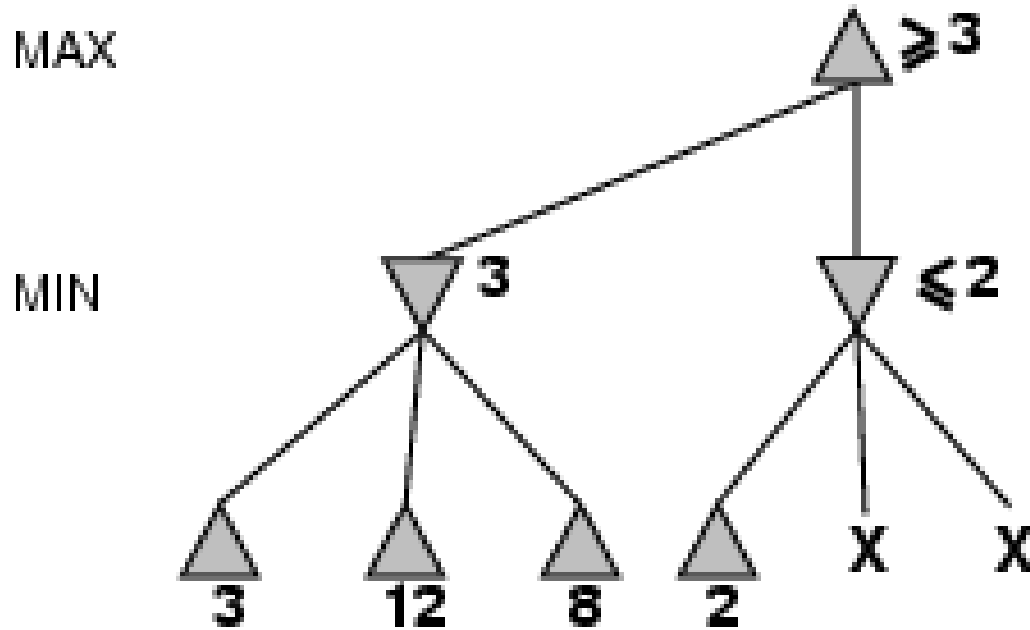
Properties of minimax

- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ (depth-first exploration)
- For chess, $b \approx 35$, $m \approx 100$ for "reasonable" games
→ exact solution completely infeasible

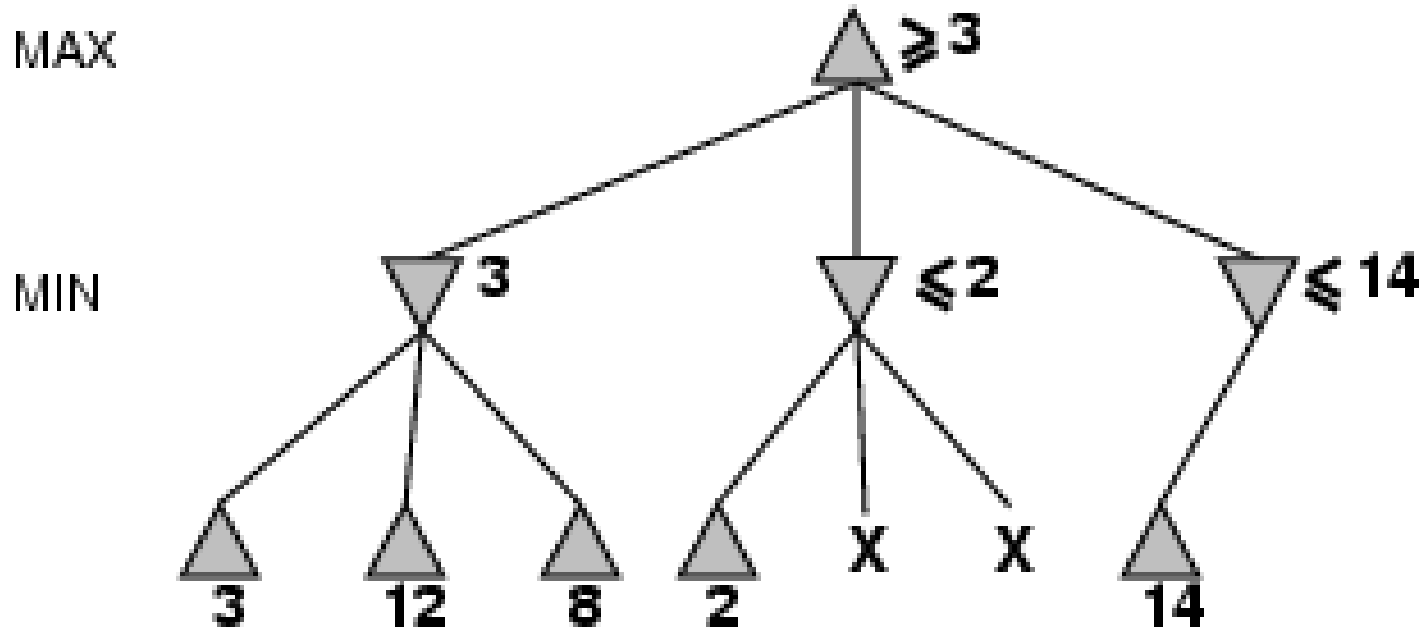
α - β pruning example



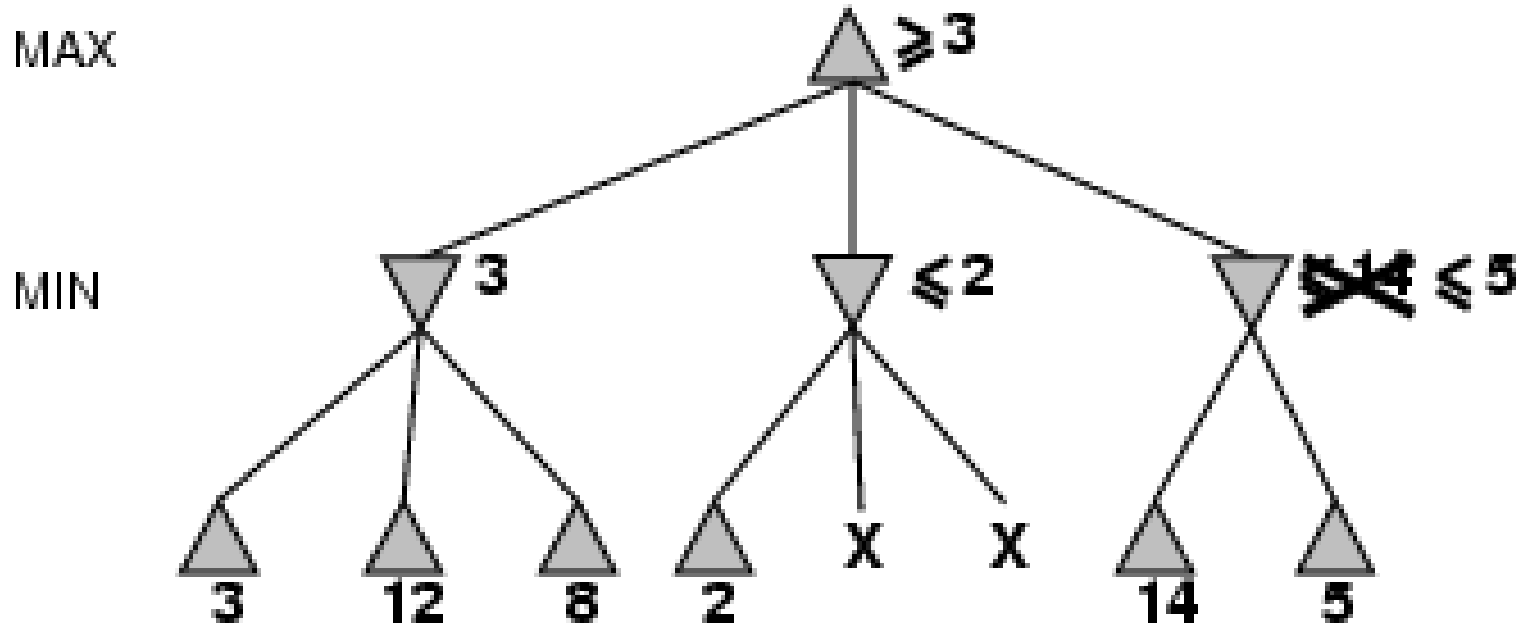
α - β pruning example



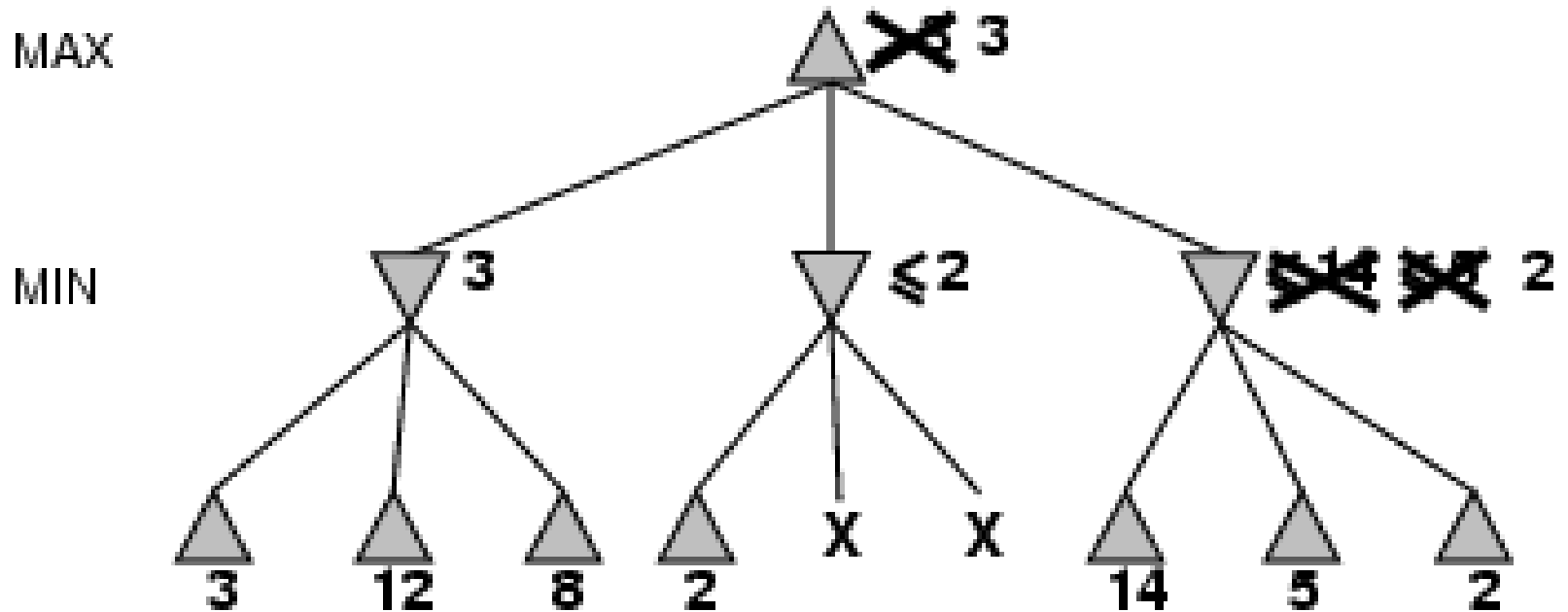
α - β pruning example



α - β pruning example



α - β pruning example



Properties of α - β

- Pruning **does not** affect final result
- Good move ordering improves effectiveness of pruning
- With "perfect ordering" time complexity = $O(b^{m/2})$
 - **doubles** depth of search

Why is it called α - β ?

- α is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for *max*
- If v is worse than α , *max* will avoid it
→ prune that branch
- Define β similarly for *min*

MAX

MIN

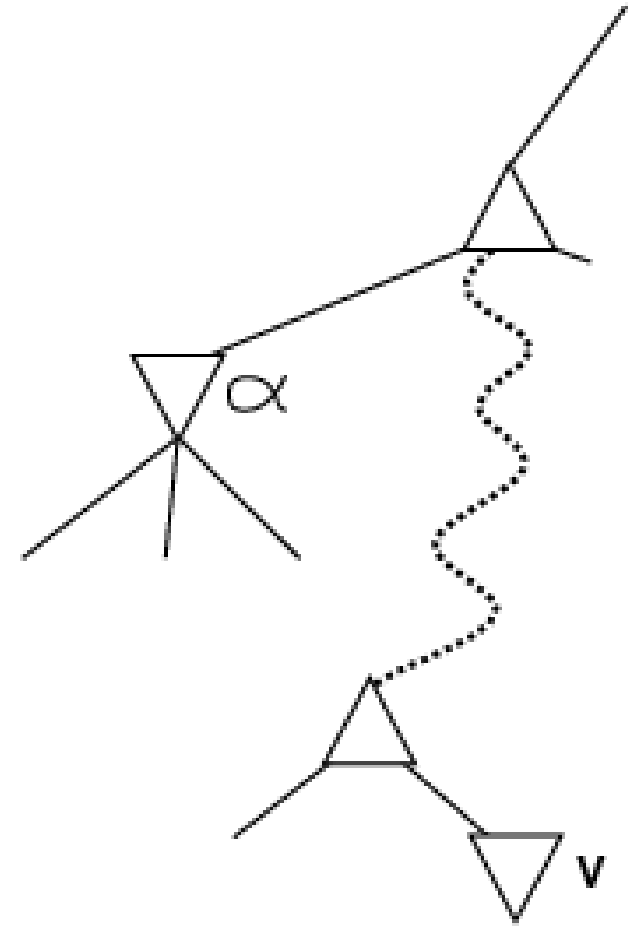
..

..

..

MAX

MIN



The α - β algorithm

```
function ALPHA-BETA-SEARCH(state) returns an action  
   $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$   
  return the action in  $\text{ACTIONS}(\text{state})$  with value  $v$ 
```

```
function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  if  $\text{TERMINAL-TEST}(\text{state})$  then return  $\text{UTILITY}(\text{state})$   
   $v \leftarrow -\infty$   
  for each  $a$  in  $\text{ACTIONS}(\text{state})$  do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$   
    if  $v \geq \beta$  then return  $v$   
     $\alpha \leftarrow \text{MAX}(\alpha, v)$   
  return  $v$ 
```

```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  if  $\text{TERMINAL-TEST}(\text{state})$  then return  $\text{UTILITY}(\text{state})$   
   $v \leftarrow +\infty$   
  for each  $a$  in  $\text{ACTIONS}(\text{state})$  do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$   
    if  $v \leq \alpha$  then return  $v$   
     $\beta \leftarrow \text{MIN}(\beta, v)$   
  return  $v$ 
```

Evaluation functions

- For chess, typically **linear** weighted sum of **features**

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- e.g., $w_1 = 9$ with
 $f_1(s) = (\text{number of white queens}) - (\text{number of black queens})$, etc.

Планирование

Основы планирования

Планированием называется процесс выработки последовательности действий, позволяющих достичь цели.

Среда называется средой *классического планирования*, если она является:

- полностью наблюдаемой;
- детерминированной;
- конечной;
- статической (изменения происходят только в результате действий агента);
- дискретной (сточки зрения времени, действий, объектов и результатов).

В отличие от этого, *неклассическое* планирование предназначено для частично наблюдаемых или стохастических вариантов среды и для него требуется другой набор алгоритмов и проектов агентов.

Задача планирования

Рассмотрим ситуацию, когда обычный агент, решающий задачи с помощью стандартных алгоритмов поиска (поиска в глубину, поиска A^* и т.д.), сталкивается с крупными задачами реального мира.

Сложность 1.

Агент, решающий задачи, может быть просто подавлен огромным количеством действий, не относящихся к делу.

Задача планирования

Рассмотрим задачу покупки одного экземпляра англоязычного издания настоящей книги с названием «AI: A Modern Approach» в электронном книжном магазине.

Предположим, что агент-покупатель должен совершить одно действие, связанное с покупкой, в расчете на каждый возможный десятицифровой номер ISBN, что приводит к общему количеству действий, равному 10 миллиардам. В ходе применения алгоритма поиска агент должен исследовать состояния результатов всех 10 миллиардов действий, чтобы определить, какое из них соответствует цели, заключающейся в том, чтобы приобрести экземпляр книги с номером ISBN 0137903952.

С другой стороны, разумный планирующий агент должен быть способным проработать процедуру покупки в обратном направлении, от явного описания цели, такого как Have(ISBN0137903952), и непосредственно сформировать действие Buy(ISBN0137903952). Для этого агенту требуется иметь общие знания о том, что действие Buy(x) приводит к результату Have(x).

При наличии этих знаний и цели планировщик может определить в единственном шаге унификации, что правильным действием является Buy(ISBN0137903952).

Задача планирования

Сложность 2.

Определение хорошей эвристической функции.

Пусть цель агента – купить четыре разных книги. Количество планов только для четырех этапов покупки будет составлять 10^{40} , поэтому поиск без точной эвристики даже нет смысла рассматривать.

Для человека хорошей эвристической оценкой для стоимости состояния является количество книг, которые еще предстоит купить;

Для агента, решающего задачи, это не столь очевидно, поскольку он рассматривает процедуру проверки цели как "черный ящик", который возвращает истину или ложь в ответ на каждое состояние. Поэтому агент, решающий задачи, не обладает автономностью; он требует, чтобы человек предоставлял ему эвристическую функцию для каждой новой задачи.

С другой стороны, если планирующий агент имеет доступ к явному представлению цели как конъюнкции подцелей, то может использовать единственную эвристику, независимую от проблемной области, — количество невыполненных конъюнктов. Для задачи покупки книг цель будет представлять собой выражение $\text{Have}(A) \wedge \text{Have}(B) \wedge \text{Have}(C) \wedge \text{Have}(D)$, а состояние, содержащее выражение $\text{Have}(A) \wedge \text{Have}(C)$, будет иметь стоимость 2. Таким образом, агент автоматически получает правильную эвистику для этой задачи и для многих других.

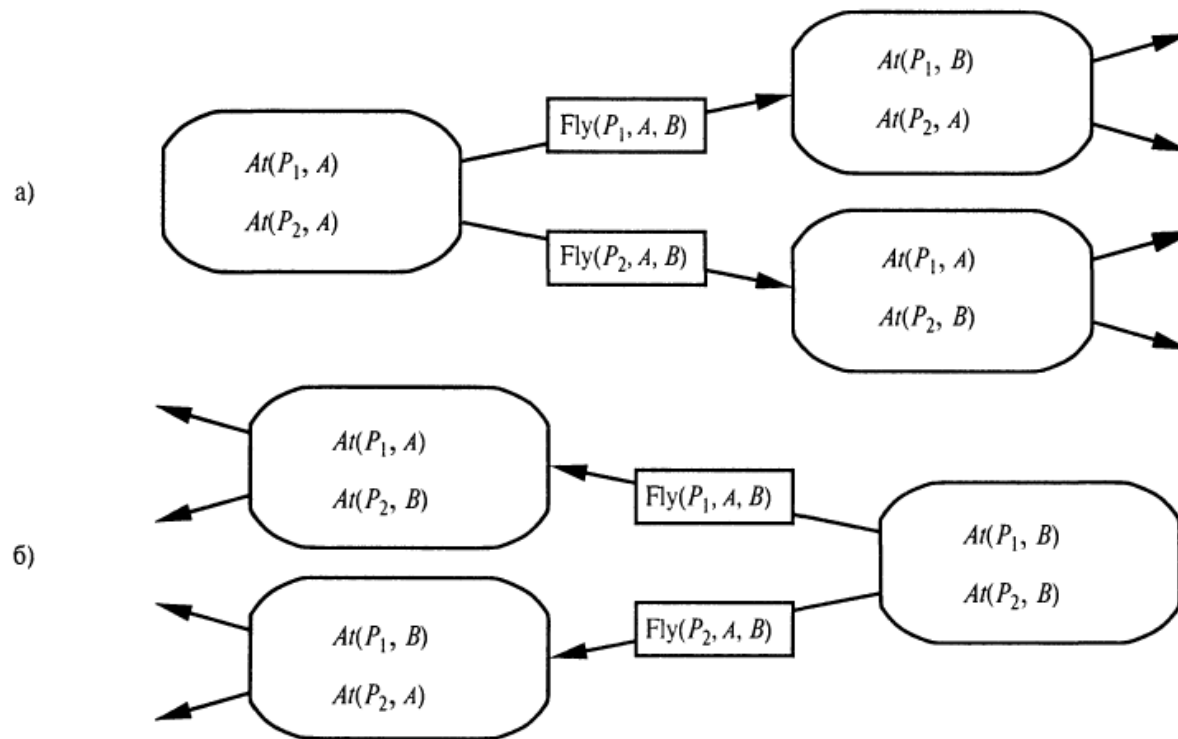
Задача планирования

Сложность 3.

Неспособность к *декомпозиции задачи*.

Например, задача доставки множества пакетов почты по соответствующим адресатам, которые разбросаны по всей Австралии. В данном случае имеет смысл найти аэропорт, ближайший к каждому адресату, и разделить общую задачу на несколько подзадач, по одной на каждый аэропорт. А в самом множестве пакетов, доставляемых через каждый конкретный аэропорт, можно определить в зависимости от города адресата допустимость дальнейшей декомпозиции.

Планирование с помощью поиска в пространстве состояний



а) прямой (прогрессивный) поиск

б) обратный (регрессивный) поиск в пространстве состояний

Прямой поиск в пространстве состояний

Формулировка задач планирования в виде задач поиска в пространстве состояний:

- Начальное состояние
- Все действия, применимые в некотором состоянии
- Проверка цели
- Стоимость этапа (обычно =1).

Пример с 10 аэропортами, в каждом имеются 5 самолетов и 20 ед. груза. Цель – переместить все грузы из аэропорта A в B .

Обратный поиск в пространстве состояний

Основное преимущество – позволяет рассматривать только **релевантные** действия. Действие релевантно конъюнктивной цели, если оно достигает одного из конъюнктов данной цели.

Пример с 10 аэропортами, в каждом имеются 5 самолетов и 20 ед. груза. Цель – переместить все грузы из аэропорта A в B .

$At(C1, B) \wedge At(C2, B) \wedge \dots \wedge At(C20, B)$

Рассматривая конъюнкт $At(C1, B)$ и двигаясь в обратном направлении, можно найти действия, имеющие этот результат. Таковым является только одно действие: $Unload(C1, p, B)$, где самолет p не задан.

Кроме этого, имеются множество нерелевантных действий, способных привести к целевому состоянию.

Какие???

В рассматриваемой задаче грузовых воздушных перевозок имеется около 1000 действий, ведущих в прямом направлении из начального состояния, но только 20 действий, позволяющих перейти в обратном направлении от цели.

Обратный поиск в пространстве состояний

Поиск в обратном направлении называют *регрессивным* планированием.

Основной вопрос регрессивного планирования – есть ли такие состояния, из которых применение некоторого действия приводит к цели?

Вычисление описаний таких состояний называется **регрессией** цели через действие.

Пример. Цель $At(C1, V) \wedge At(C2, V) \wedge \dots \wedge At(C20, V)$ и релевантное действие $Unload(C1, p, V)$, которое позволяет достичь первого конъюнкта.

Соответствующее действие будет выполнимо, только если выполнены его предусловия. Поэтому любое состояние-преемник должно включать эти предусловия: $In(C1, p) \wedge At(p, V)$. Более того, подцель $At(C1, V)$ не должна быть истинной в состоянии-преемнике (нерелевантные действия).

Поэтому описание состояния-преемника будет:

$$In(C1, p) \wedge At(p, V) \wedge At(C2, V) \wedge \dots \wedge At(C20, V)$$

Кроме выполнения требования, чтобы действия достигали некоторого желаемого литерала, должно также соблюдаться требование, чтобы действия не отменяли какие-либо желаемые литералы.

Любое действие, удовлетворяющее этому ограничению, называется **совместимым**.

Например, действие $Load(C2, p)$ не будет совместимым с текущей целью, поскольку оно отрицает литерал $At(C2, V)$.

Обратный поиск в пространстве состояний

Формирование преемников для обратного поиска.

Допустим, что при наличии описания цели G имеется действие A , которое является релевантным и совместимым. Соответствующий преемник может быть определен следующим образом:

- Любые положительные результаты действия A , которые появляются в цели G , удаляются.
- Добавляется каждый литерал предусловия A , если он еще не присутствует в определении действия.

Для осуществления обратного поиска могут использоваться любые из стандартных алгоритмов поиска. Завершение работы происходит после выработки такого описания преемника, которое соответствует начальному состоянию задачи планирования.

В случае использования логики первого порядка для обеспечения соответствия с начальным состоянием может потребоваться подстановка переменных в описание преемника.

Например, после подстановки $\{p/ P12\}$: $In(C1,P12) \wedge At(P12,B) \wedge At(C2,B) \wedge \dots \wedge At(C20, B)$
Затем данная подстановка должна быть применена к действиям, ведущим из этого состояния к цели, что приводит к получению решения [$Unload(C1, P12, B)$].

Планирование с частичным упорядочением

Недостаток прямого и обратного поиска в пространстве состояний – рассматриваются только строго линейные последовательности действий, непосредственно связанные с начальным или целевым состоянием, что не позволяет воспользоваться преимуществами декомпозиции задачи.

Более предпочтительным является подход, позволяющий работать над несколькими подцелями независимо, достигать их с помощью нескольких субпланов, а затем объединять эти субпланы. Еще одно преимущество такого подхода – большая гибкость при определении последовательности, в которой составляется окончательный план, т.е. планировщик вначале может работать над "очевидными" или "важными" решениями, не будучи вынужденным прорабатывать все этапы в хронологическом порядке.

Например, планирующий агент, который находится в Беркли и желает попасть в Монте-Карло, может вначале попытаться найти рейс из Сан-Франциско в Париж; получив информацию о времени отправления и прибытия этого рейса, он может затем заняться поиском способов того, как доехать и выехать из соответствующих аэропортов.

Общая стратегия, в которой в процессе поиска выбор определенных этапов откладывается на более позднее время, называется стратегией с **наименьшим вкладом** (least commitment).

Пример задачи с надеванием пары туфель

Goal(RightShoeOn $\wedge$ LeftShoeOn)

Init()

Action(RightShoe, Precond: RightSockOn, Effect: RightShoeOn)

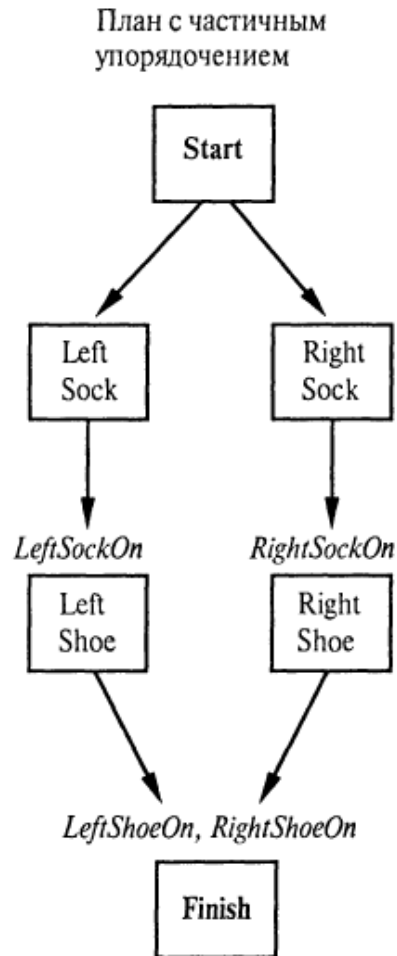
Action(RightSockEffect: RightSockOn)

Action(LeftShoe, Precond: LeftSockOn, Effect: LeftShoeOn)

Action(LeftSock, Effect: LeftSockOn)

Планирование с частичным упорядочением

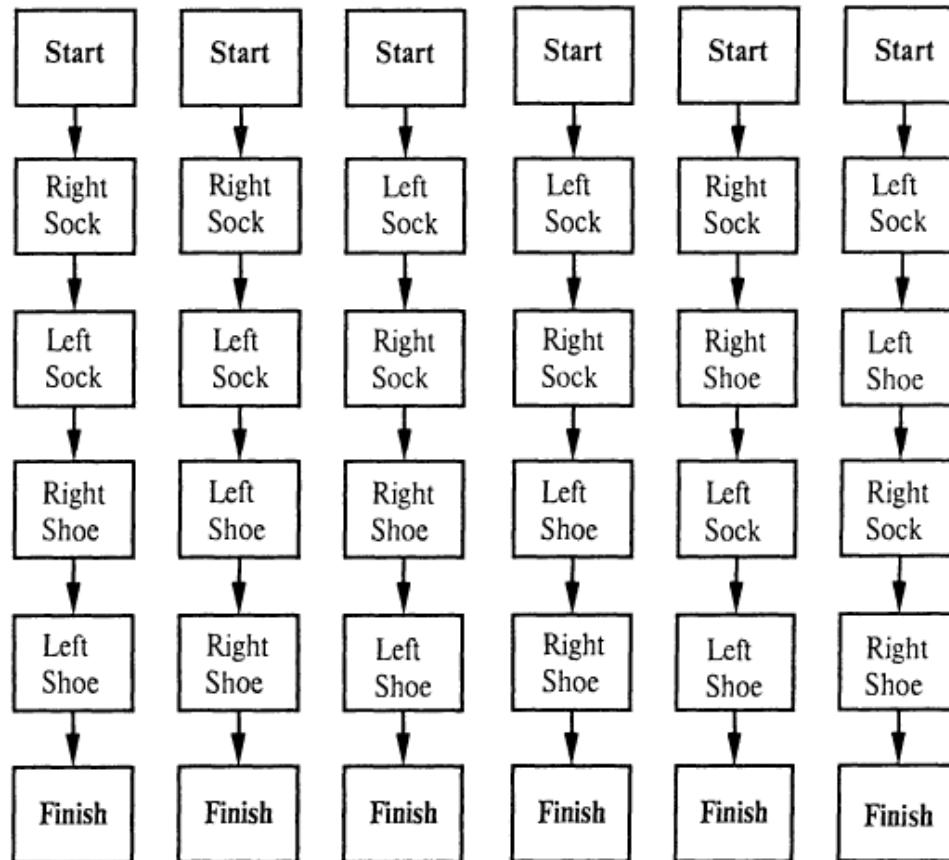
Любой алгоритм планирования, способный включить в план два действия без указания того, какое из них должно быть выполнено первым, называется **планировщиком с частичным упорядочением** (partial-order planner).



Линеаризация плана с частичным упорядочением

Решение с частичным упорядочением соответствует шести возможным планам с полным упорядочением; каждый из них называется **линеаризацией** плана с частичным упорядочением.

Планы с полным упорядочением



Планирование с частичным упорядочением

Планирование с частичным упорядочением может быть реализовано в виде поиска в пространстве планов с частичным упорядочением (далее просто "планами»).

Поиск начинается с пустого плана. После этого рассматриваются способы уточнения плана до тех пор, пока не удастся составить полный план, который решает данную задачу. Действия, рассматриваемые в этом поиске, являются не действиями в мире, а действиями в планах: добавление в план этапа; наложение упорядочения, согласно которому одно действие должно занять место перед другим; и т.д.

Состояниями рассматриваемой задачи поиска являются планы (в основном незаконченные).

Компоненты плана

- Множество **действий**, из которых состоят этапы плана. Эти действия берутся из множества действий в задаче планирования. "Пустой" план содержит только действия Start и Finish. Действие Start не имеет предусловий, а его результатом являются все литералы в начальном состоянии задачи планирования. Действие Finish не имеет результатов, а его предусловиями являются литералы цели в задаче планирования.
- Множество **ограничений упорядочения**. Каждое ограничение упорядочения находится в форме $A \prec B$, которая читается как "А перед В" и означает, что действие А должно быть выполнено в какое-то время перед действием В, но не обязательно непосредственно перед ним. Ограничения упорядочения должны описывать правильный вариант частичного упорядочения. Любой цикл (такой как $A \prec B$ и $B \prec A$) представляет противоречие, поэтому ни одно ограничение упорядочения не может быть добавлено в план, если оно создает цикл.

Компоненты плана

- Множество **причинных связей**. Причинная связь между двумя действиями A и B в плане записывается как $A \xrightarrow{p} B$ (“ A достигает p для B ”).

Например, в следующей причинной связи $RightSock \xrightarrow{RightSockOn} RightShoe$ утверждается, что $RightSockOn$ (надет правый носок) представляет собой результат действия $RightSock$ и предусловие действия $RightShoe$. В ней также содержится утверждение, что предусловие $RightSockOn$ должно оставаться истинным со времени действия $RightSock$ до времени действия $RightShoe$. Другими словами, план не может быть дополнен путем добавления какого-либо нового действия C , которое конфликтует с причинной связью.

Действие C конфликтует со связью $A \xrightarrow{p} B$, если C имеет результат $\neg p$ и если C может (в соответствии с ограничениями упорядочения) происходить после A и перед B . Некоторые авторы называют причинные связи интервалами защиты, поскольку связь $A \xrightarrow{p} B$ защищает предусловие p от его отрицания в интервале от A до B .

- Множество **открытых предусловий**. Предусловие является открытым, если оно не достигнуто с помощью какого-то действия в плане. Планировщики действуют по принципу сокращения множества открытых предусловий до пустого множества без внесения противоречия.

Пример задачи с надеванием пары туфель

Действия: {*RightSock*, *RightShoe*, *LeftSock*, *LeftShoe*, *Start*, *Finish*}
Упорядочения: {*RightSock* < *RightShoe*, *LeftSock* < *LeftShoe*}
Связи: { *RightSock* $\xrightarrow{\text{RightSockOn}}$ *RightShoe*, *LeftSock* $\xrightarrow{\text{LeftSockOn}}$ *LeftShoe*,
 RightShoe $\xrightarrow{\text{RightShoeOn}}$ *Finish*, *LeftShoe* $\xrightarrow{\text{LeftShoeOn}}$ *Finish*}
Открытые предусловия: {}

Согласованный план

Согласованный план (consistent plan) – план, в котором не имеется циклов в ограничениях упорядочения и нет конфликтов с причинными связями. Согласованный план без открытых предусловий представляет собой **решение**.

Формулировка задачи планирования

- Начальный план содержит действия Start и Finish, ограничение упорядочения $\text{Start} -< \text{Finish}$, не включает ни одной причинной связи и имеет все предусловия в действии Finish в качестве открытых предусловий.
- Функция определения преемника произвольным образом выбирает одно открытое предусловие p действия B и вырабатывает план-преемник для каждого возможного согласованного способа выбора действия A , которое достигает p . Согласованность обеспечивается принудительно следующим образом.
 - Причинная связь $A \xrightarrow{p} B$ и ограничение упорядочения $A -< B$ добавляются к плану. Действие A может быть существующим действием в плане или новым действием. Если оно является новым, добавить его к плану, а также добавить $\text{Start} -< A$ и $A -< \text{Finish}$.
 - Разрешаются конфликты между новой причинной связью и всеми существующими действиями, а также между действием A (если оно является новым) и всеми существующими причинными связями. Конфликт между $A \xrightarrow{p} B$ и C разрешается путем обеспечения того, чтобы действие C происходило в какое-то время за пределами интервала защиты, либо за счет добавления ограничения $B -< C$, либо путем добавления ограничения $C -< A$. Для одного из них или обоих добавляются состояния-преемники, если они приводят к созданию согласованных планов.

Формулировка задачи планирования

- В ходе проверки цели осуществляется проверка того, является ли текущий план решением первоначальной задачи планирования. Поскольку вырабатываются только согласованные планы, в проверке цели достаточно только проконтролировать, есть ли еще открытые предусловия.

Пример планирования с частичным упорядочением

Пример задачи замены колеса со стертой покрышкой

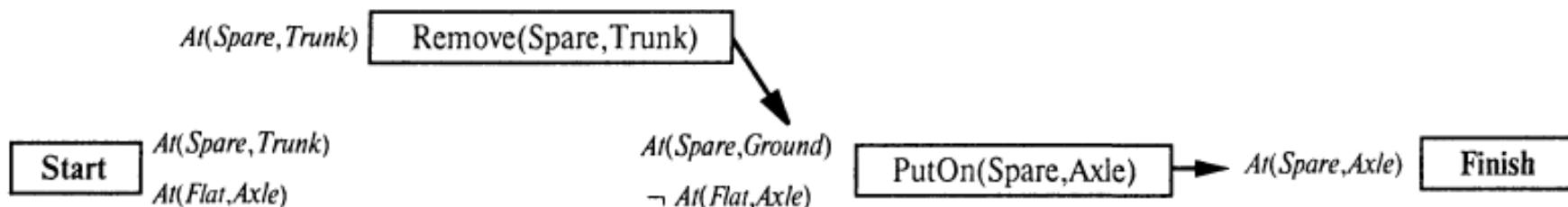
```
Init(At(Flat,Axle) ∧ At(Spare,Trunk))
Goal(At(Spare,Axle))
Action(Remove(Spare,Trunk),
      Precond: At(Spare,Trunk)
      Effect:  $\neg$ At(Spare,Trunk) ∧ At(Spare,Ground))
Action(Remove(Flat,Axle),
      Precond: At(Flat,Axle)
      Effect:  $\neg$ At(Flat,Axle) ∧ At(Flat,Ground))
Action(PutOn(Spare, Axle),
      Precond: At(Spare,Ground) ∧ ¬ At(Flat,Axle)
      Effect:  $\neg$ At(Spare,Ground) ∧ At(Spare,Axle))
Action(LeaveOvernight,
      Precond:
      Effect:  $\neg$ At(Spare,Ground) ∧ ¬At(Spare,Axle) ∧ ¬At(Spare,Trunk)
              $\wedge$   $\neg$ At(Flat,Ground) ∧ ¬At(Flat,Axle))
```

Пример планирования с частичным упорядочением

Поиск решения начинается с начального плана, содержащего действие Start с результатом $At(Spare, Trunk) \wedge At(Flat, Axle)$ и действие Finish с единственным предусловием $At(Spare, Axle)$.

Затем вырабатываются преемники путем взятия открытого предусловия для его проработки (не допускающей отмены) и выбора среди возможных действий для его достижения.

1. Взять единственное открытое предусловие, $At(Spare, Axle)$, действия Finish. Выбрать единственное применимое действие, $PutOn(Spare, Axle)$.
2. Взять предусловие $At(Spare, Ground)$ действия $PutOn(Spare, Axle)$. Выбрать единственное применимое действие, $Remove(Spare, Trunk)$, чтобы достичь его.

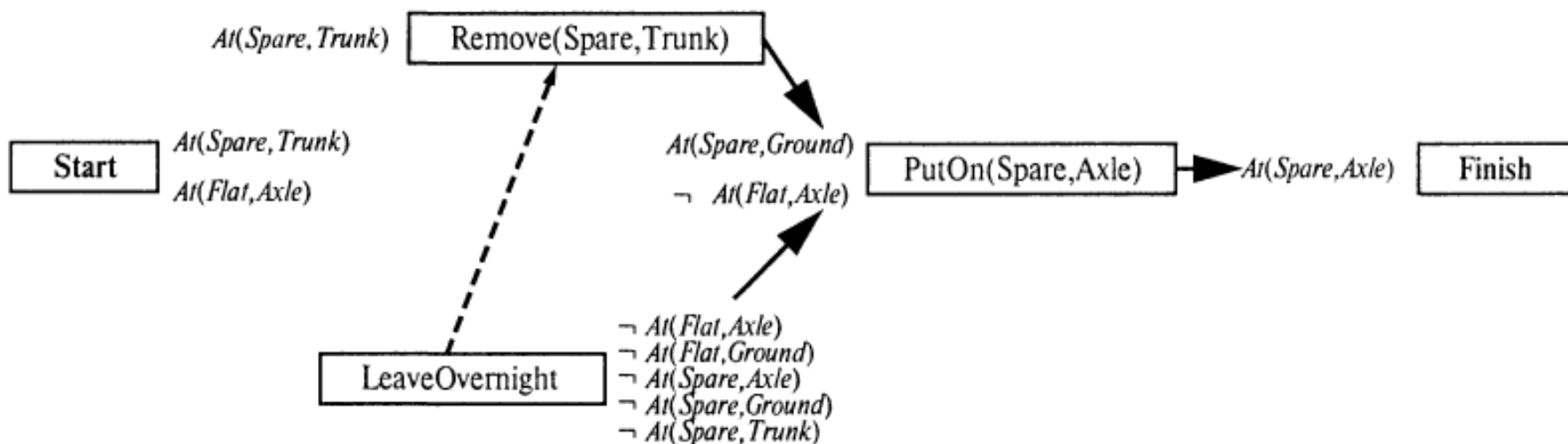


Пример планирования с частичным упорядочением

3. Взять предусловие $\neg \text{At}(\text{Flat}, \text{Axle})$ действия $\text{PutOn}(\text{Spare}, \text{Axle})$. Просто для того чтобы поступить вопреки здравому смыслу, выберем действие LeaveOvernight , а не действие $\text{Remove}(\text{Flat}, \text{Axle})$. Действие LeaveOvernight также имеет результат $\neg \text{At}(\text{Spare}, \text{Ground})$, который означает, что оно конфликтует со следующей причинной связью:

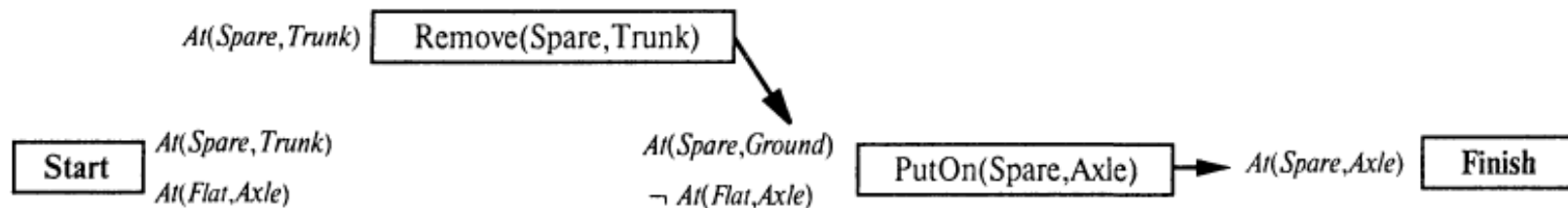
$\text{Remove}(\text{Spare}, \text{Trunk}) \xrightarrow{\text{At}(\text{Spare}, \text{Ground})} \text{PutOn}(\text{Spare}, \text{Axle})$

Чтобы разрешить этот конфликт, добавим ограничение упорядочения, которое помещает действие LeaveOvernight перед действием $\text{Remove}(\text{Spare}, \text{Trunk})$.



Пример планирования с частичным упорядочением

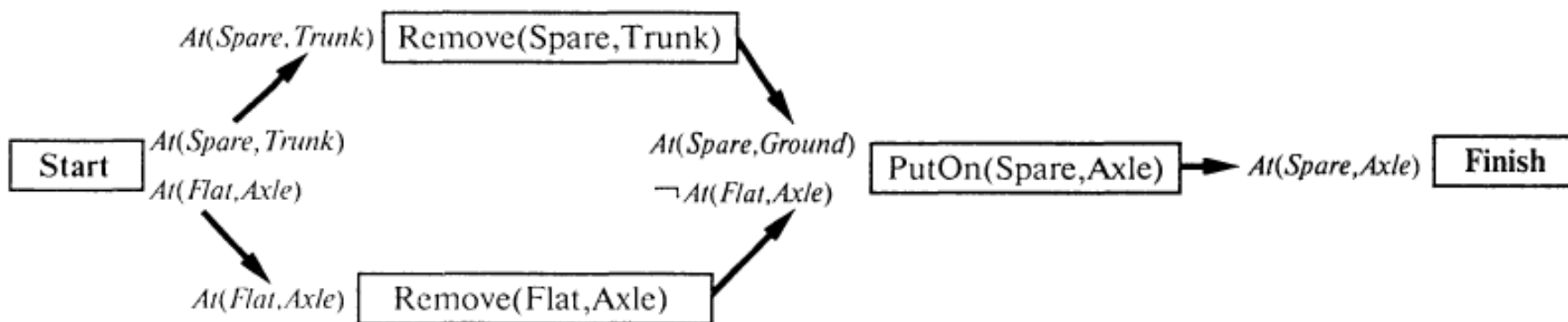
4. В этот момент единственным оставшимся открытым предусловием является предусловие $At(Spare, Trunk)$ действия $Remove(Spare, Trunk)$. Единственным действием, позволяющим достичь этого предусловия, является существующее действие $Start$, но причинная связь от $Start$ к $Remove(Spare, Trunk)$ конфликтует с результатом $\neg At(Spare, Trunk)$ действия $LeaveOvernight$. В данный момент не существует способа разрешить конфликт с действием $LeaveOvernight$ — его нельзя переупорядочить таким образом, чтобы оно находилось перед $Start$ (поскольку ни одно действие не может происходить перед действием $Start$), и нельзя переупорядочить его так, чтобы оно находилось после $Remove(Spare, Trunk)$ (поскольку уже имеется ограничение, которое упорядочивает его перед $Remove(Spare, Trunk)$). Поэтому необходимо вернуться к одному из предыдущих состояний, удалить действие $LeaveOvernight$ и две последние причинные связи, возвратившись в состояние



Планировщик фактически доказал, что действие $LeaveOvernight$ не может применяться в качестве способа замены колеса.

Пример планирования с частичным упорядочением

5. Еще раз рассмотрим предусловие $\neg \text{At}(\text{Flat}, \text{Axle})$ действия $\text{PutOn}(\text{Spare}, \text{Axle})$. На этот раз выберем действие $\text{Remove}(\text{Flat}, \text{Axle})$.
6. Снова возьмем предусловие $\text{At}(\text{Spare}, \text{Trunk})$ действия $\text{Remove}(\text{Spare}, \text{Trunk})$ и выберем действие Start , чтобы его достичь. На сей раз конфликты не возникают.
7. Возьмем предусловие $\text{At}(\text{Flat}, \text{Axle})$ действия $\text{Remove}(\text{Flat}, \text{Axle})$ и выберем действие Start для его достижения. Это дает нам полный согласованный план или решение.



Планирование действий в пространстве задач

При решении сложных задач пользуются разбиением на подзадачи.

Цель разбиения – прийти к решению совокупности элементарных (решающихся за 1 шаг перебора в пространстве состояний или решения которых известны) задач.

Описание задачи в пространстве состояний – тройка (S, F, G):

- S - множество начальных состояний,
- F - множество операторов, отображающих описание состояний,
- G - множество целевых состояний.

Планирование действий в пространстве задач

Пример Маккарти. Ханойские башни.

3 колышка (1 2 3) и 3 диска (А,В,С).

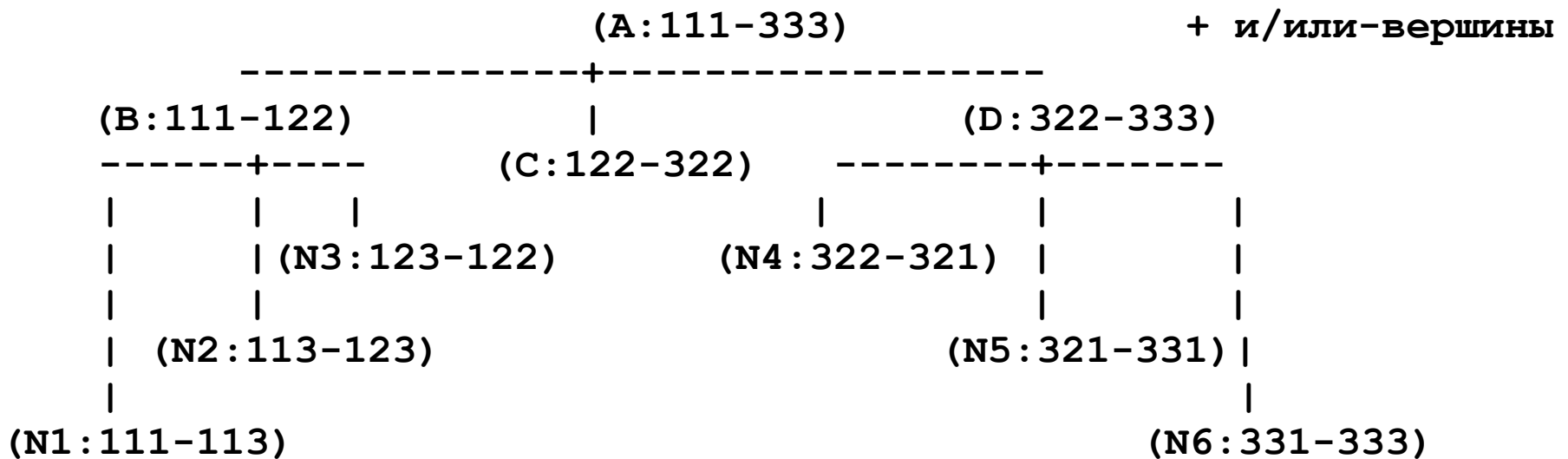
3 подзадачи:

A		A	(1)	(1, 1, 1) -> (1, 2, 2)	(B)
B	->	B	Имя: (A) (2)	(1, 2, 2) -> (3, 2, 2)	Имя: (C)
C	--	C	(3)	(3, 2, 2) -> (3, 3, 3)	(D)
1 2 3		1 2 3.			

Задача (2)-элементарная, (1) и (3) следует свести к совокупности элементарных.

В практических целях представления задач выполняют на графоподобных структурах.

Планирование действий в пространстве задач



Заключительные вершины соответствуют элементарным задачам.

Применение ключевых операторов

Пусть (S, F, G) – описание исходной задачи в пространстве состояний,
 $g_1, \dots, g_n$ – последовательность промежуточных состояний, которые удалось выделить.
 Необходимо свести задачу (S, F, G) к множеству подзадач $(S, F, \{g_1\}), (\{g_1\}, F, \{g_2\}), \dots, (\{g_n\}, F, G)$. (1)
 Различают 2 случая:

- Состояние g_i ($i = 1 \div n$) определяется явно $\Rightarrow$ подзадачи решаются в произвольном порядке.
- g_i ($i = 1 \div n$) определяется неявно, например, $\exists$ множество G_i ($i = 1 \div n$) $\forall$ элемент может служить в качестве основного промежуточного состояния g_i . Тогда задача (S, F, G_i) должна быть решена раньше чем $(\{g_i\}, F, G_{i+1})$.

Решая задачу в пространстве состояний, требуется определить решающую цепочку операторов.
 Нахождение всей цепочки – сложная задача, но всегда можно выделить необходимый шаг решения.
 Для определения основного промежуточного состояния можно пользоваться так называемыми
 «ключевыми операторами» (КО).

Обозначим $f \in F$ КО для задачи (S, F, G) .

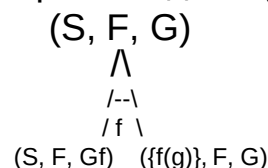
Пусть Gf - множество состояний, к которым применим f .

Тогда первой следующей за (S, F, G) задачей будет (S, F, Gf) .

Решая ее определяют основное состояние $g \in Gf$, и тогда можно сформулировать элементарную задачу:
 $(\{g\}, F, \{f(g)\})$, где $f(g)$ = состояние, достигаемое применением f к g . (*)

Т.о. осталось решить задачу $(\{f(g)\}, F, G)$.

Графическое представление сведения задачи к подзадачам с использованием КО
 (без указания элементарной задачи (*)):



На следующем шаге находят КО задачи (S, F, Gf) и т.д.

В результате – последовательность промежуточных состояний $g_1, \dots, g_n$.

Задача об обезьяне и банане (I)

А - первоначальное нахождение обезьяны.

В - первоначальное нахождение ящика.

С - точка над которой к потолку подвешены бананы.

- А, В, С – точки в горизонтальной плоскости 3D-евклидова пространства.

Предполагается, что обезьяна может :

- Подойти к ящику.
- Переместить ящик в точку С.
- Взобраться на ящик.
- Сорвать бананы.

Задача об обезьяне и банане (2)

Решение.

Очевидно, $F = \{f_1, f_2, f_3, f_4\}$.

Действия и условия применимости операторов задаются в виде продукций:

$f_i(x_1, x_2, x_3, x_4) \rightarrow (y_1, y_2, y_3, y_4)$, где $\rightarrow$ моделирует действие.

- x_1 – положение обезьяны (на полу).
- x_2 – { 1: обезьяна на ящике, 0: не на ящике.
- x_3 – положение ящика (на полу).
- x_4 – { 1: обезьяна сорвала бананы, 0: нет.

$S = \{(A, 0, B, 0)\},$

$G = \{(C, 1, C, 1)\}.$

Задача об обезьяне и банане (3)

Конструируем правила переписывания:

- Подойти(y_1) $f_1(x_1, 0, x_3, x_4) \rightarrow (y_1, 0, x_3, x_4)$
- Передвинуть(α) $f_2(x_1, 0, x_1, x_4) \rightarrow (\alpha, 0, \alpha, x_4)$
- Взобраться $f_3(x_1, 0, x_1, x_4) \rightarrow (x_1, 1, x_1, x_4)$
- Сорвать $f_4(x_1, 1, x_1, 0) \rightarrow (x_1, 1, x_1, 1)$

Описание исходной задачи имеет вид: $(\{(A, 0, B, 0)\}, F, \{(C, 1, C, 1)\})$

Зададим связи между возможными КО и различиями:

- Если $(x_1 \neq B)$ то применить f_1 .
- Если $(x_3 \neq C)$ то применить f_2 .
- Если $(x_2 \neq 1)$ то применить f_3 .
- Если $(x_4 \neq 1)$ то применить f_4 .

Задача об обезьяне и банане (4)

Шаг 1. Вычислить различия для исходной задачи.

Основной признак целевого состояния $x_4 = 1$. В списке $\{(A, 0, B, 0)\}$: $x_4 = 0$.

Уменьшить это различие может оператор $f_4 \Rightarrow f_4 - KO$.

Для исходной задачи получим пару подзадач:

$$(\{(A, 0, B, 0)\}, F, G)$$
$$\wedge$$
$$/--\backslash$$
$$/ f_4 \backslash$$
$$(\{(A, 0, B, 0)\}, F, Gf_4) \quad (\{f_4(S1)\}, F, G)$$

где $S1 \in Gf_4$ – множеству состояний, к которым применим оператор f_4 .

Решая левую задачу найдем $S1$ – состояние, получаемое в результате решения левой подзадачи.

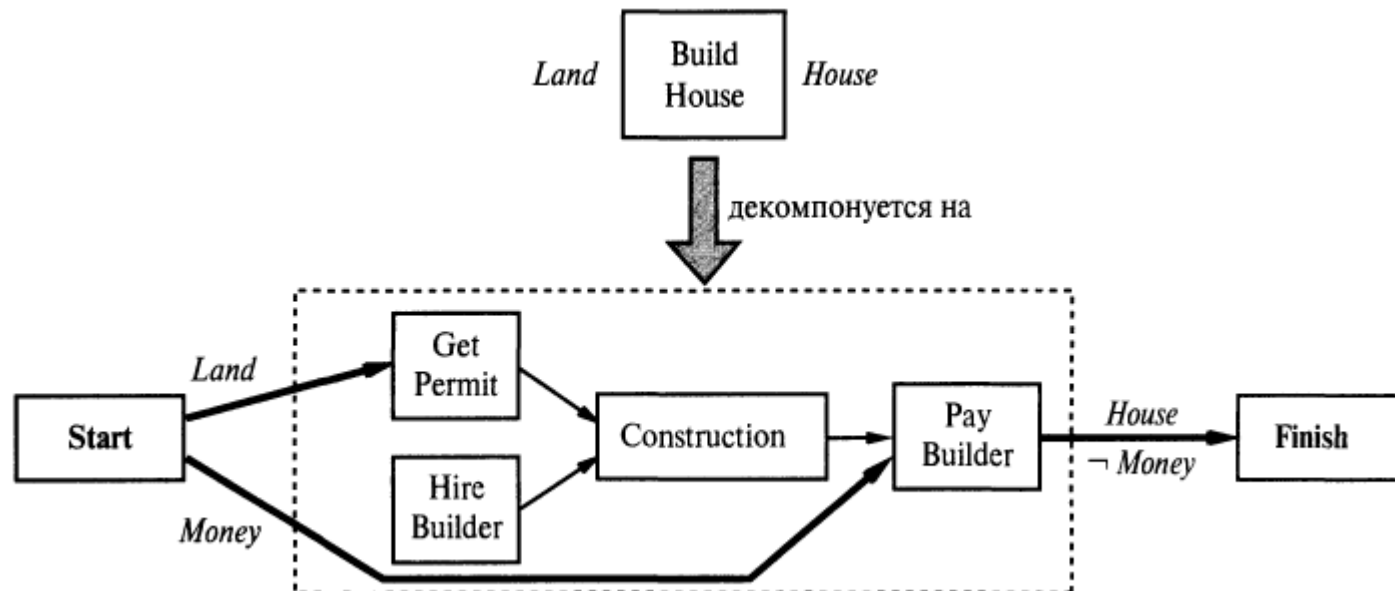
Состояние из Gf_4 описывается условиями:

- обезьяна в точке C.
- ящик в точке C.
- Обезьяна на ящике.

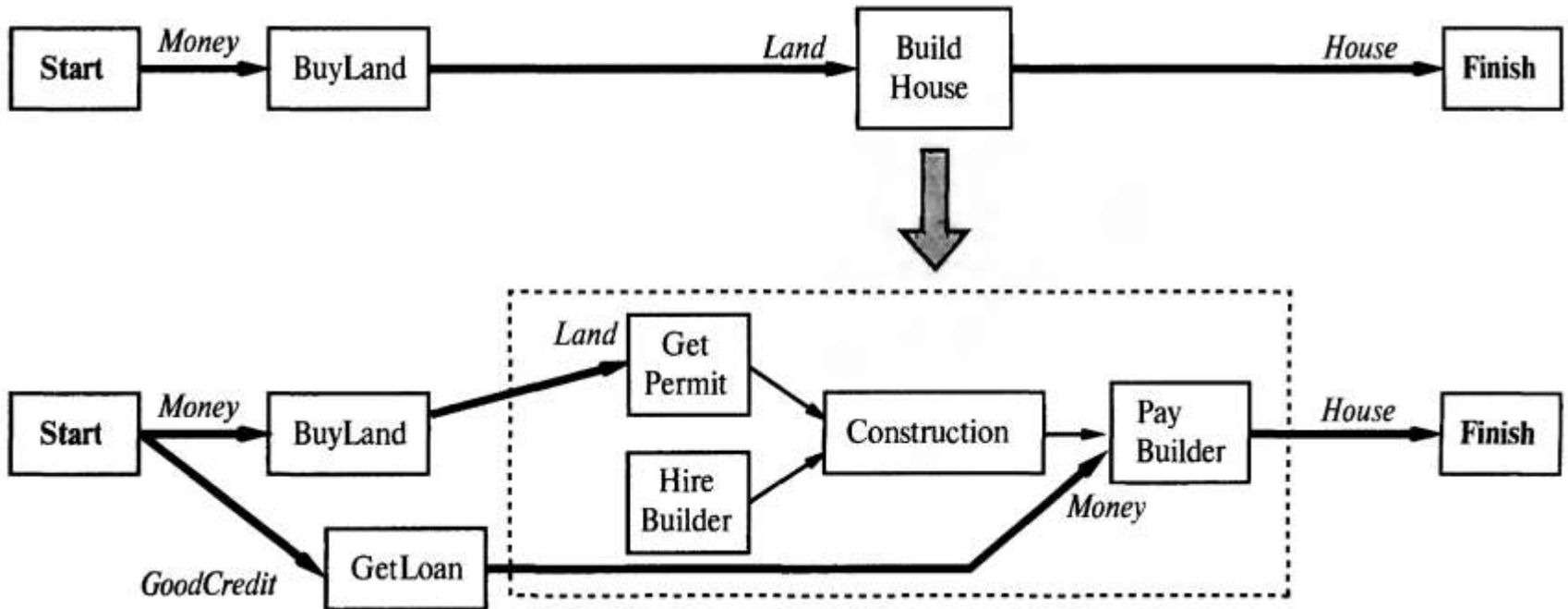
Шаги 2,3,4:... - самостоятельно.

Планирование иерархической сети задач

Метод планирования **HTN** (Hierarchical Task Network) рассматривает первоначальный план, который описывает задачу, как описание на очень высоком уровне того, что должно быть сделано, например строительство дома. Планы уточняются путем применения декомпозиций действий. В каждой декомпозиции действия одно действие высокого уровня сводится к частично упорядоченному множеству действий низкого уровня. Поэтому в декомпозициях действий воплощены знания о том, как осуществляются действия. Процесс продолжается до тех пор, пока в плане не остаются только примитивные действия (могут быть выполнены агентом автоматически).



Планирование иерархической сети задач



Мультиагентное планирование

Задача игры в парный теннис. Два агента играют в одной команде и могут присутствовать в одном из четырех мест:

- [Left,Baseline] (слева, у задней линии),
- [Right, Baseline] (справа, у задней линии),
- [Left, Net] (слева, под сеткой) и
- [Right, Net] (справа, под сеткой).

Мяч может быть отбит, если в нужном месте находится один и только один игрок

Цель – отбить мяч и обеспечить, чтобы хотя бы один из них играл под сеткой.

Agents(A, B)

Init(*At*(A, [Left, Baseline]) $\wedge$ *At*(B, [Right, Net]) $\wedge$
 Approaching(Ball, [Right, Baseline])) $\wedge$ *Partner*(A,B) $\wedge$ *Partner*(B,A)
Goal(*Returned*(Ball) $\wedge$ *At*(agent, [x, Net]))

Action(*Hit*(agent, Ball),

Precond: *Approaching*(Ball, [x, y]) $\wedge$ *At*(agent, [x, y]) $\wedge$
 Partner(agent,partner) $\wedge$ $\neg$ *At*(partner, [x,y]),
 Effect: *Returned*(Ball))

Action(*Go*(agent, [x, y]),

Precond: *At*(agent, [a, b]),
 Effect: *At*(agent, [x,y]) $\wedge$ $\neg$ *At*(agent, [a, b]))

Мультиагентное планирование

Решением мультиагентной задачи планирования является **совместный план** (joint plan), состоящий из действий для каждого агента. Совместный план представляет собой решение, если цель будет достигнута при условии, что каждый агент выполнит назначенные ему действия. Решением данной задачи игры в теннис является план:

Plan 1:

A: [Go(A, [Right, Baseline]), Hit(A, Ball)]

B: [NoOp(B), NoOp(B)]

Если оба агента имеют одинаковую базу знаний и если это решение является единственным, то все сложится идеально; каждый из агентов сможет определить это решение, а затем совместно выполнить его. Но к большому сожалению для этих агентов существует другой план, позволяющий так же успешно достичь цели, как и первый:

Plan 2:

A: [Go(A, [Left, Net]), NoOp(A)]

B: [Go(B, [Right, Baseline]), Hit(B, Ball)]

Мультиагентное планирование

Решение проблемы – **координация**

Механизмы координации

Простейший метод, с помощью которого группа агентов может обеспечить координацию при выполнении совместного плана, состоит в принятии определенного **соглашения** (convention) до начала совместной деятельности. Соглашением является любое ограничение, касающееся выбора совместных планов, выходящее за рамки основного ограничения, в соответствии с которым совместный план должен работать, если ему следуют все агенты. Например, соглашение "придерживаться своей стороны поля" станет причиной того, что партнеры в парном теннисе выберут план 2, а соглашение, что "один игрок всегда остается у сетки", приведет их к плану 1.

Некоторые соглашения, такие как вождение по правильной стороне дороги, приняты настолько широко, что считаются **общественными законами**. Естественные языки также могут рассматриваться как соглашения.

Более общий подход состоит в использовании соглашений, независимых от проблемной области. Если каждый агент эксплуатирует один и тот же алгоритм планирования с одними и теми же входными данными, то может следовать соглашению о выполнении первого найденного осуществимого плана и быть уверенным в том, что другие агенты сделают такой же выбор. Более надежная, но и более дорогостоящая стратегия – выработать все совместные планы, а затем выбрать, тот из них, внешнее представление для вывода на печать которого находится на первом месте в алфавитном порядке.

Механизмы координации

Модель поведения птицеподобного агента (который иногда именуется орнитоидом или боидом от слова boid, или birdoid) выполняет три перечисленных ниже правила, применяя определенный метод их комбинирования:

1. **Отделение.** Направлять свой полет подальше от соседей, если ты начинаешь к ним слишком заметно приближаться.
2. **Соединение в линию.** Придерживаться направления полета, позволяющего занять среднюю позицию по отношению к соседям.
3. **Выравнивание.** Рулить в сторону средней ориентации (направления движения) соседей.

Если все птицы выполняют одни и те же описанные выше правила, то вся стая обнаруживает **эмерджентное поведение** (от слова emergent— появляющийся), совершая полет в виде одного псевдоустойчивого тела приблизительно с постоянной плотностью, которое не рассеивается со временем.

Механизмы координации

- При отсутствии применимого соглашения агенты могут использовать общение для получения общих знаний об осуществимом совместном плане. Например, в парном теннисе игрок может крикнуть "Мой!" или "Твой!", имея в виду мяч, чтобы указать на предпочтительный для него совместный план.
- Один игрок может неявно сообщить о предпочтительном совместном плане другому, просто выполнив его первую часть. В нашей задаче игры в теннис, если агент А направился к сетке, то агент В обязан отойти назад, к линии подачи, чтобы отбить мяч, поскольку план 2 является единственным совместным планом, который начинается с того, что агент А направляется к сетке. Такой подход к координированию действия, иногда называемый **распознаванием плана** (plan recognition), является применимым, если для безошибочного определения нужного совместного плана достаточно одного действия (или краткой последовательности действий).

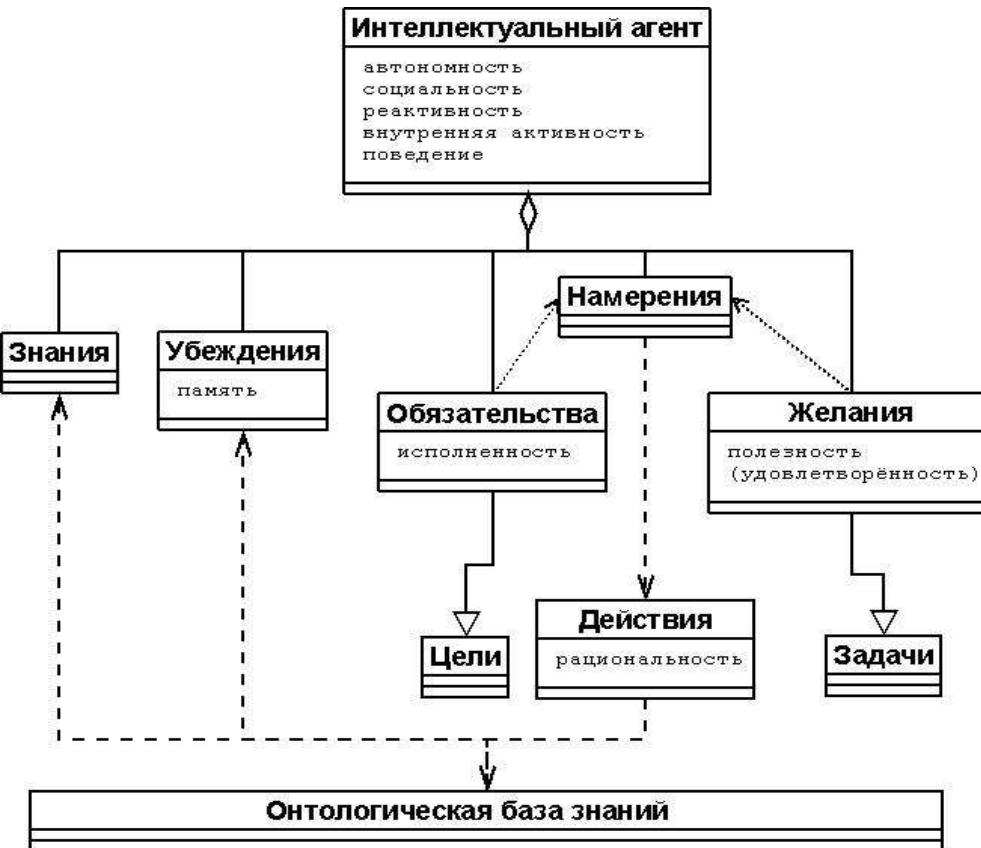
Многоагентная система — это система, в которой несколько взаимодействующих интеллектуальных агентов пытаются совместно достичь некоторый набор целей или выполнить некоторый набор задач (“Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence”, MIT, 1999)

Свойства интеллектуального агента

Агенты — это активные объекты (программные модули), которые могут инициировать целенаправленную деятельность по восприятию среды и воздействию на неё.

Интеллектуальные агенты обладают следующими «ментальными» свойствами (или их подмножеством):

- **знания (knowledge)** — постоянные, неизменяемые в процессе функционирования знания агента о себе, среде и других агентах;
- **убеждения (beliefs)** — знания агента о среде (в том числе, о других агентах), которые могут стечением времени изменяться и становиться неверными;
- **желания (desires)** — состояния, которых агент желает достичь (могут быть противоречивыми), аналогичны *целям*;
- **намерения (intentions)** — действия, которые собирается выполнить вследствие своих желаний или в силу взятых на себя обязательств;
- **обязательства (commitments)** — задачи, решение которых агент берет на себя в рамках кооперации с другими агентами по их просьбе или поручению.



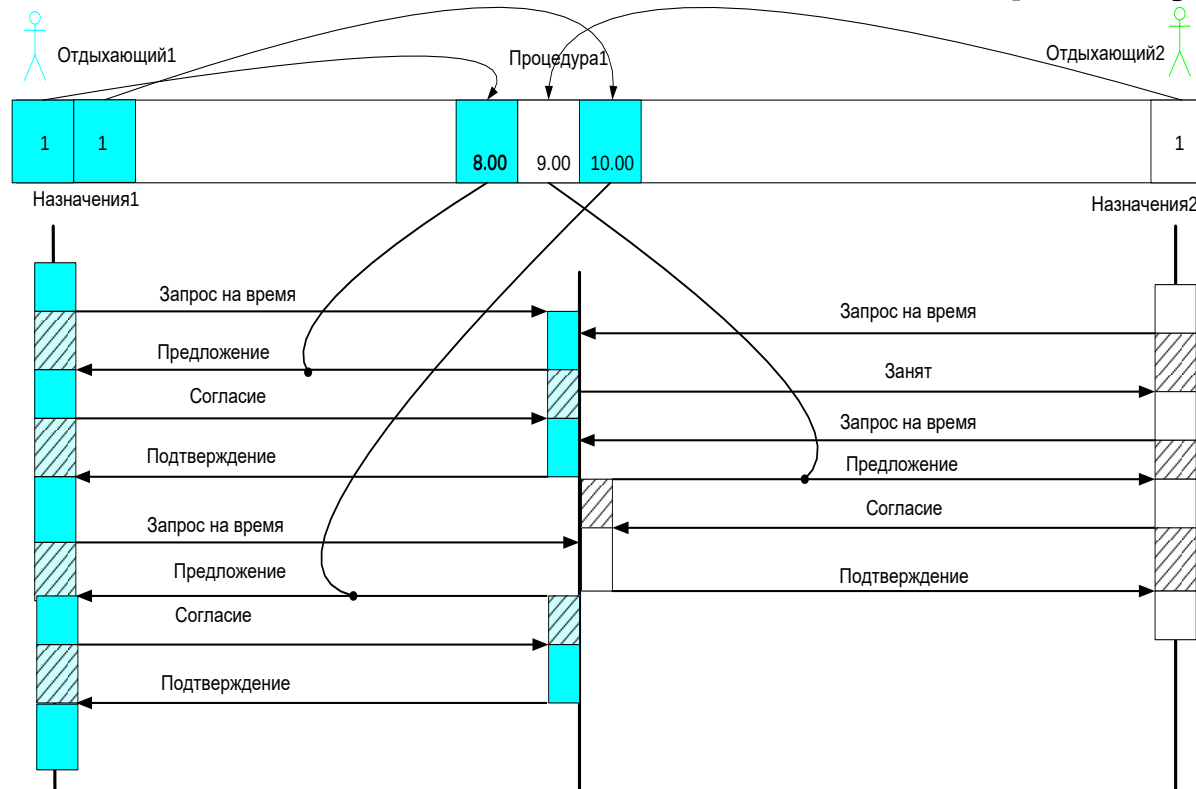
Принципы конструирования агента на основе спецификаций FIPA



FIPA (FOUNDATION FOR INTELLIGENT PHYSICAL AGENTS) –
Международная федерация по разработке интеллектуальных
физических агентов

Пример взаимодействия агентов (1):

отдыхающим назначена одна и та же процедура



Типы операций принятия решений:

для агента-Отдыхающего: проверка незанятости, проверка ограничений, проверка времени отдыха

для агента-Процедуры: поиск свободного времени, проверка отправителя

Затраты на коммуникативные процессы:

для агента-Отдыхающего: запрос на время, Согласие (Несогласие)

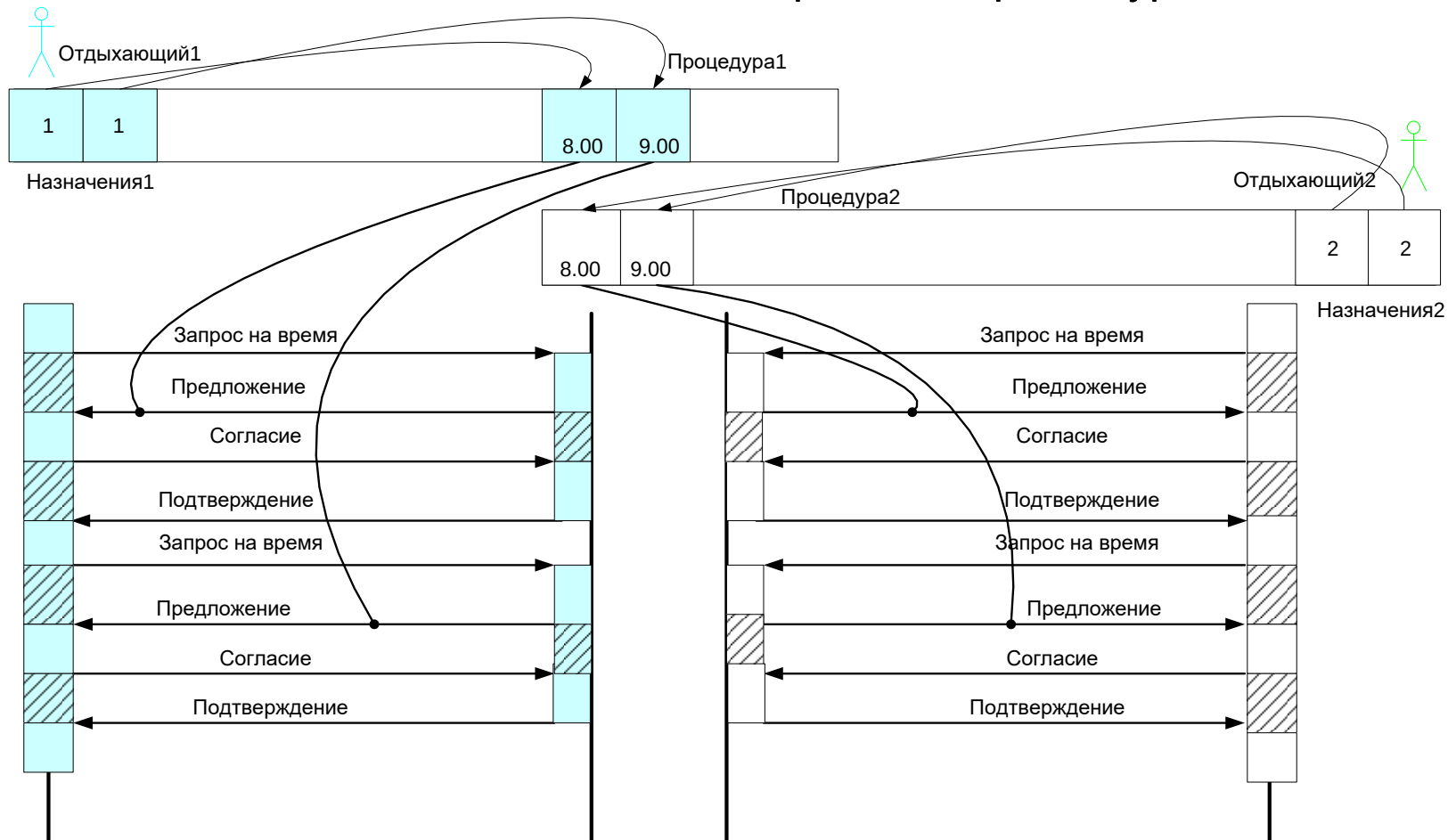
для агента-Процедуры: предложение (в случае, если есть свободное время), подтверждение соглашения

Входные данные:

Для отдыхающего № 1 назначено две процедуры типа Процедура № 1. Для отдыхающего № 2 назначена одна процедура того же типа.

График работы процедурного кабинета: 8.00-9.00, 9.00-10.00, 10.00-11.00

Пример взаимодействия агентов (2): отдыхающим назначены разные процедуры



Входные данные:

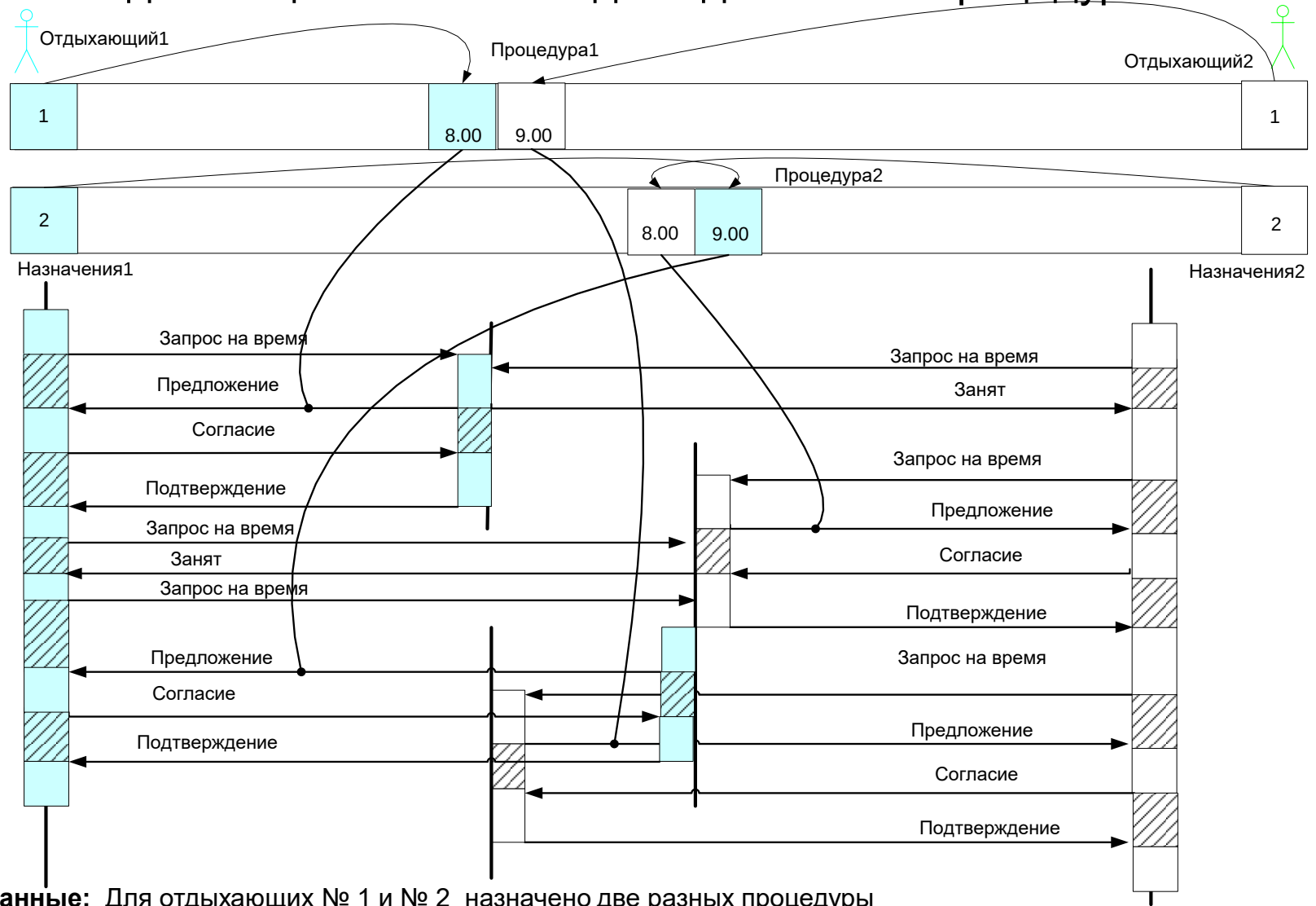
Для отдыхающего № 1 назначено две процедуры типа Процедура № 1. Для отдыхающего № 2 назначено две процедуры типа Процедура № 2.

Графики работ процедурных кабинетов:

Процедура №1 8.00-9.00, 9.00-10.00

Процедура №2 8.00-9.00, 9.00-10.00

Пример взаимодействия агентов (3): отдыхающим назначены две одинаковые процедуры



Входные данные: Для отдыхающих № 1 и № 2 назначено две разных процедуры типа Процедура № 1 и Процедура № 2.
 Графики работ процедурных кабинетов:
 Процедура №1 8.00-9.00, 9.00-10.00
 Процедура №2 8.00-9.00, 9.00-10.00

FIPA - протокол взаимодействия между агентами

